

Highlights

- Homotopy-guided RL (R2SP) tackles sparse-reward 1v1 6-DoF WVR air combat.
- Traj-level reward relabelling spreads event rewards via linear/Gaussian kernels.
- Progressive sub-task guidance learns delayed, coupled ele-ail-rud-throttle.

Homotopy-Guided Reinforcement Learning for Sparse-Reward WVR Air Combat

Abstract

Within-visual-range (WVR) air combat is a challenging benchmark for autonomous decision-making, where agents must jointly learn high-dimensional flight-control and tactical policies under sparse, delayed and adversarial rewards. Existing deep reinforcement learning (DRL) approaches typically alleviate these difficulties by (i) simplifying the engagement geometry or flight dynamics, (ii) introducing dense reward shaping based on handcrafted geometric advantage functions (e.g., angle or distance superiority), or (iii) cascading a high-level decision-maker with a low-level flight controller that underestimates tight elevator–aileron–rudder–throttle coupling and throttle-to-airspeed delay. However, such designs either sacrifice aerodynamic and tactical fidelity, bias learning towards specific heuristic advantage measures, or struggle to handle tightly coupled three-dimensional manoeuvring, so that sparse event-based rewards and credit assignment remain inadequately addressed and training in realistic simulators is still sample-inefficient and unstable. This paper proposes a homotopy-guided reinforcement learning framework for WVR air combat, named R2SP (Reward Relabelling and Sub-task Progressing), which explicitly targets these shortcomings. At the tactical level, R2SP introduces a trajectory-level reward-relabelling mechanism that redistributes sparse event and outcome rewards over time via linear and Gaussian kernels. The per-step rewards are derived from the timing and type of combat events, rather than manually engineered geometric advantage functions, enabling effective reuse of partial successes and near-miss trajectories for credit assignment. At the execution level, R2SP employs a progressive sub-task control scheme that decomposes three-dimensional manoeuvring into guided sub-tasks for pitch, heading and speed, while a performance-triggered scheduling-and-rollback mechanism coordinates these sub-tasks under heterogeneous chan-

*These authors are the corresponding authors.

¹These authors contributed equally to this work.

nel dynamics. Experiments in a high-fidelity WVR air-combat simulator show that R2SP achieves higher win rates, faster convergence and more accurate flight control than several state-of-the-art DRL baselines and rule-based tactics, and generates manoeuvres consistent with pilot tactics. These results indicate that trajectory-level reward relabelling and progressive sub-task control provide an effective route to tackling sparse-reward adversarial tasks that couple high-level planning with low-level continuous control.

Keywords: Within-visual-range air combat, Trajectory-level reward relabelling, Progressive sub-task control, Deep reinforcement learning, Modeling and simulation

1. Introduction

Within-visual-range (WVR) air combat remains a central element of modern air operations, demanding visual confirmation, high-agility manoeuvres and rapid tactical decision-making (HU et al., 2025). Even in environments where beyond-visual-range (BVR) engagements are possible, factors such as rules of engagement, situational uncertainty and electronic warfare frequently lead to close-in encounters (Qian et al., 2024). WVR air combat therefore continues to be both a core pilot competency and a critical testbed for autonomous decision-making systems.

WVR engagements can be modelled as high-dimensional, sparse-reward Markov decision processes (MDPs) in which an agent must reason over coupled kinematics, weapon envelopes and adversarial behaviour. The joint observation space combines ownship and adversary states, radar information and missile parameters, aggravating the curse of dimensionality. The opponent’s policy evolves during training, creating a non-stationary, game-theoretic environment that departs from the independent and identically distributed (i.i.d.) assumption underlying many deep reinforcement learning (DRL) algorithms. Rewards are extremely sparse and delayed (Zhou et al., 2023): meaningful feedback mainly occurs at critical events such as target lock-on, missile launch, missile evasion or kill confirmation, while most intermediate actions yield no explicit signal (ZHANG et al., 2024). At the same time, low-level flight control is tightly coupled across channels—elevator, aileron, rudder and throttle must be coordinated (De Marco et al., 2023), and throttle commands affect airspeed with substantial delay and in turn modulate the effectiveness of other control surfaces. These structural properties collectively make WVR air combat a particularly demanding testbed for DRL.

To mitigate these structural difficulties, existing DRL-based WVR air combat methods have largely evolved along two distinct methodological lines: advantage-based reward engineering and hierarchical policy decomposition. The first line, advantage-based reward engineering, manually constructs dense reward terms from geometric relationships such as aspect angle, heading crossing angle and relative distance, providing frequent feedback and accelerating early learning, but also embedding strong heuristic priors that bias the policy towards predefined “good” geometries and potentially suppress tactically novel behaviours (Cheng et al., 2021). The second line, hierarchical policy decomposition, separates high-level combat decisions from low-level flight control to reduce the effective decision space (Pope et al., 2023) and enable specialised algorithms at each level; however, the high-level layer still depends on sparse, adversarial event outcomes for credit assignment, and the low-level layer must handle coupled three-dimensional dynamics and throttle delays without explicitly exploiting heterogeneous channel characteristics.

To address these limitations, we propose a homotopy-guided RL framework for WVR air combat, R2SP, which jointly reshapes the temporal distribution of sparse combat-event rewards and the low-level control objective along a reward–control homotopy, so that high-level tactics and low-level manoeuvring can be learned along smoother optimisation paths. At the tactical level, R2SP replaces handcrafted geometric advantage functions with a trajectory-level reward-relabelling mechanism that redistributes sparse event- and outcome-based rewards along the temporal dimension using linear and Gaussian kernels. The relabelled rewards are computed from the timing and type of environment events instead of additional angle- or distance-based shaping terms, enabling effective reuse of partial successes and near-miss trajectories for credit assignment. At the control level, R2SP exploits the heterogeneous characteristics of flight-control channels by decomposing the three-dimensional manoeuvring task into progressive pitch, heading and speed sub-tasks, and by using a performance-triggered scheduling mechanism that gradually morphs the low-level objective from simple to fully coupled control. R2SP acts as a data-driven, learning-based expert system that enables the transition from rule-based to learning-based tactical systems.

The main contributions of this work are summarised as follows:

- We formulate **homotopy-guided RL for sparse-reward 1-vs-1 WVR air combat**, viewing training as reward–control homotopy beyond fixed dense rewards or static hierarchical controllers.
- We instantiate this paradigm in **R2SP**, combining (i) trajectory-level reward

relabelling based on the timing and type of combat events (linear and Gaussian kernels) and (ii) progressive sub-task flight control with performance-gated scheduling and rollback over pitch, heading and speed channels.

- We provide a **comprehensive six-degree-of-freedom (6-DoF) evaluation**, including strong DRL and rule-based baselines, separate analyses of upper-level tactics and lower-level control, and multi-generation self-play and human–AI engagements, showing consistent gains in sample efficiency, control accuracy and win rates with tactically meaningful, iteratively evolving manoeuvres.

2. Related Work

2.1. Traditional air-combat decision-making

Early research on air-combat decision-making mainly relied on analytical and rule-based methods, including differential-game formulations, optimal guidance laws, finite-state machines and expert-system style rule sets (Xu et al., 2022). These approaches provide interpretable strategies and can encode pilot knowledge explicitly, but they often assume simplified engagement geometries, fixed opponent behaviours or low-dimensional state spaces (Shan et al., 2022). As a result, their performance can degrade in highly dynamic WVR scenarios with complex manoeuvres, realistic 6-DoF dynamics and learning opponents (Chai et al., 2023).

2.2. Deep reinforcement learning for WVR air combat

With the rapid development of DRL, a growing body of work has explored DRL-based WVR air combat decision-making. Existing methods can be broadly grouped into four lines.

Environment simplification. One line of research employs reduced-order aircraft models and abstract continuous actions such as overload, thrust or roll angle instead of full Euler-angle dynamics and detailed missile models (Wang et al., 2024; Qiu et al., 2020). These abstractions ease policy learning by reducing the dimensionality of the state-action space and smoothing the dynamics, and can yield reasonable policies under the simplified assumptions. However, the resulting agents operate in environments that omit important aerodynamic effects and weapon constraints, so their tactics may fail to transfer to high-fidelity simulators where coupled 6-DoF dynamics and realistic missile envelopes play a critical role.

Hybrid knowledge integration. Another line designs hybrid architectures that integrate domain knowledge into the learning loop, for example by combining

neural policies with rule-based tactical modules (Xu et al., 2019), cognitive architectures with working and long-term memory, or deep generative models trained on expert or evolutionarily generated trajectories (Hu et al., 2021; Wang et al., 2016; Zhou et al., 2020). These methods leverage prior knowledge to stabilise learning under non-stationary opponents and sparse feedback, but they typically depend on specific expert tactics or scenario configurations. When facing unseen manoeuvre styles or qualitatively different engagement geometries, the embedded priors can limit adaptability and lead to overfitting to the original training distribution.

Advantage-based reward engineering. A large number of DRL-based WVR studies alleviate reward sparsity by handcrafting dense reward terms from geometric relationships, such as angle-of-attack, aspect angle, heading crossing angle, relative distance and altitude differences (Zhou et al., 2023; ZHANG et al., 2024). Such advantage-based rewards provide frequent feedback and can significantly accelerate early learning, especially when combined with self-play. However, they also embed strong assumptions about what constitutes a “good” engagement geometry. This tends to bias the learned policies towards conservative or human-crafted tactics, and may suppress alternative manoeuvres that are effective but not directly aligned with the chosen advantage measures.

Hierarchical policy decomposition. A fourth line adopts hierarchical DRL frameworks that separate high-level air-combat decisions (e.g., target selection, missile launch and coarse guidance) from low-level attitude or waypoint tracking controllers (Xu et al., 2025). This decomposition reduces the effective decision space at each level and allows specialised algorithms to be applied to tactical planning and control (Pope et al., 2021). Nonetheless, high-level policies in these frameworks are still trained under sparse, delayed and adversarial rewards (ZHANG et al., 2024), while low-level controllers are often required to handle tightly coupled control channels and throttle delays without explicit mechanisms that exploit their heterogeneous dynamics.

Overall, existing approaches either hard-code geometric advantage functions into the reward or rely on generic hierarchical controllers that do not reshape the learning path itself. In contrast, our homotopy-guided RL for WVR air combat (R2SP) explicitly designs a reward–control homotopy: trajectory-level reward relabelling replaces handcrafted geometric advantage terms for high-level policies, while progressive sub-task control gradually morphs the low-level objective from simple surrogates to fully coupled 6-DoF manoeuvring.

3. Model formulation

3.1. WVR air combat problem statement

The outcome of WVR air combat is usually determined within a short period of time, so variations in fuel mass and related parameters can be neglected over the course of a single fight. The model assumes constant gravitational acceleration and ignores the effects of the Earth’s rotation and curvature on flight trajectories. To capture the weapon and sensor characteristics that shape realistic WVR tactics, the environment further incorporates high-fidelity infrared missile and fire-control radar models, whose kinematic, guidance and detection equations are detailed in Appendix C (missile modeling) and Appendix D (radar modeling).

For simulation, we use a high-fidelity 6-DoF aircraft model controlled by throttle, elevator, aileron and rudder. Realistic turning, climbing and diving require tightly coordinated use of these surfaces, while throttle changes affect air-speed with noticeable delay and alter the aerodynamic conditions for other controls. This strong cross-channel coupling and delayed speed response make precise control difficult and aggravate temporal credit assignment.

At the tactical level, WVR air combat requires complex real-time decisions under partial and noisy observations, including target detection, missile employment and potential cooperation. Rewards arise mainly from critical events such as radar lock-on, missile launch or evasion, and aircraft loss, resulting in an extremely sparse signal. To handle these distinct characteristics, we decompose the task into a higher-level air combat decision-making layer and a lower-level flight-control layer, each formulated as an MDP tailored to its role.

3.2. MDP model for higher-level air combat decision-making

3.2.1. State space

The state space for the higher-level decision-making layer is defined in Eq. (1), where the first term encodes the positions and velocities of friendly and enemy aircraft, the second term represents radar detection and track information, and the third term corresponds to the remaining missile count and related status variables. The complete state vector is 56-dimensional and can be written as in Eq. (1).

$$s_t^{\text{upper}} = \left[\begin{array}{c} \left(\begin{array}{c} lon_t^k, lat_t^k, h_t^k, v_t^{k,\text{east}}, v_t^{k,\text{north}}, v_t^{k,\text{up}}, \\ M_t^k, a_{x,t}^k, a_{y,t}^k, a_{z,t}^k, \cos \theta_t^k, \sin \theta_t^k, \\ \cos \varphi_t^k, \sin \varphi_t^k, \cos \gamma_t^k, \sin \gamma_t^k, \cos \alpha_t^k, \sin \alpha_t^k, \\ \cos \beta_t^k, \sin \beta_t^k, \omega_t^{k,\theta}, \omega_t^{k,\varphi}, \omega_t^{k,\gamma} \end{array} \right)_{k=1,2} \\ \left(\begin{array}{c} \cos \theta_t^b, \sin \theta_t^b, \cos \varphi_t^b, \sin \varphi_t^b, \\ l_t^b, \Delta v_{x,t}^b, \Delta v_{y,t}^b, \Delta v_{z,t}^b, \Delta M_t \end{array} \right), \quad n_{\text{missile}} \end{array} \right]. \quad (1)$$

Where $k = 1$ represents the friendly aircraft, and $k = 2$ represents the opposing aircraft. A detailed description of the notation is provided in Appendix A.

3.2.2. Action space

The action space at the higher level consists of target flight states for the controlled aircraft and a discrete missile control command. The action vector is defined as in Eq. (2) and includes:

$$a_t^{\text{upper}} = [\Delta\theta_t^b, \Delta\varphi_t^b, \Delta V_t, \xi_t^{\text{missile}}], \quad (2)$$

where $\Delta\theta_t^b$ and $\Delta\varphi_t^b$ are the required adjustments for pitch angle and heading angles in the body-fixed coordinate system; ΔV_t represents the flight speed that the agent must adjust; ξ_t^{missile} denotes the missile launch control variable, where $\xi_t^{\text{missile}} = 0$ indicates no missile launch, and $\xi_t^{\text{missile}} = 1$ indicates missile launch. Whether a missile launch results in a hit depends on the weapon employment conditions; further details are provided in Appendix C and Appendix D.

3.2.3. Reward function

At the higher decision-making level, the reward function exhibits strong sparsity: non-zero rewards are issued only at critical combat events. We use a small set of event rewards as light shaping around key behaviours, while letting outcome rewards dominate so that the task remains sparse and outcome-driven. As summarised in Table 1, event rewards are tied only to radar lock-on/loss, being locked, missile launch and successful evasion, and their magnitudes are intentionally much smaller than those of kill and loss rewards. Among these signals, the lock-on reward provides a low-magnitude but temporally extended incentive for maintaining a stable fire-control track, acting as a proxy for weapon opportunity without introducing explicit geometric advantage terms. A brief penalty after target loss discourages frequent track breaks and unstable radar management, supplying just enough intermediate feedback without resorting to dense angle- or distance-based shaping.

This event- and outcome-based reward structure intentionally reflects the sparse and delayed nature of WVR air combat feedback. In later sections, these sparse rewards serve as the basis for trajectory-level reward relabelling, which improves temporal credit assignment for the high-level policy.

3.3. MDP model for lower-level flight control

While the 1-vs-1 WVR engagement is modelled as a sparse-reward MDP at the higher decision-making level, the aircraft still requires an inner-loop controller

Table 1: Event and outcome rewards at the higher decision-making level.

Type	Content	Weight
Event rewards	Radar lock-on	0.1
	Target loss	-0.1
	Being locked-on	-0.1
	Missile launch	1
	Successful missile evasion	1
Outcome rewards	Confirmed kill	10
	Combat loss	-10

to track the target flight states output by the tactical policy. We therefore formulate a separate MDP for the lower-level flight-control module. This auxiliary MDP uses a dense, physically interpretable reward to encode tracking and safety objectives for the aircraft, but it does not modify the sparse event- and outcome-based rewards that determine the combat result. The primary objective of the lower-level flight control system is to realise high-precision aircraft manoeuvring so that the aircraft tracks the target flight states generated by the higher-level decision module.

3.3.1. State space

To support this objective, we define a 26-dimensional state vector, as given in Eq. (3), which contains

$$\begin{aligned}
 s_t^{\text{low}} = [& lon_t, lat_t, h_t, v_t^{\text{east}}, v_t^{\text{north}}, v_t^{\text{up}}, v_t, a_t^{x^b}, a_t^{y^b}, a_t^{z^b}, \\
 & \theta_t, \varphi_t, \gamma_t, \omega_t^{x^b}, \omega_t^{y^b}, \omega_t^{z^b}, \alpha_t, \beta_t, \Delta v_t, \omega_t^\theta, \omega_t^\varphi, \\
 & \omega_t^\gamma, \sin(\Delta\theta_t), \cos(\Delta\theta_t), \sin(\Delta\varphi_t), \cos(\Delta\varphi_t)].
 \end{aligned} \tag{3}$$

A detailed description of the notation is provided in Appendix A.

3.3.2. Action space

The action space for the lower-level controller consists of the continuous control inputs applied to the primary flight control surfaces and throttle, defined as Eq. (4):

$$a_t^{\text{low}} = [\xi_t^{\text{ele}}, \xi_t^{\text{ail}}, \xi_t^{\text{rud}}, \xi_t^{\text{thr}}]. \tag{4}$$

where ξ_t^{ele} , ξ_t^{ail} , ξ_t^{rud} , and ξ_t^{thr} represent the control inputs for the elevator, aileron, rudder, and throttle, respectively.

3.3.3. Reward function

Based on the magnitude and nature of these deviations, we define a composite reward function r_t^{low} comprising four terms: given by

$$r_t^{\text{low}} = r_t^{\text{sta}} + r_t^{\text{alt}} + r_t^{\text{env}} + r_t^{\text{att}}. \quad (5)$$

The flight-state reward r_t^{sta} is computed from the discrepancy between the current and target states and serves as the core component of the control objective; its detailed formulation is provided in Eq. (22) in Sec. 4.3. The altitude penalty term r_t^{alt} prevents the aircraft from flying below a prescribed threshold altitude. If the altitude drops below zero, the episode is terminated and a large terminal penalty is issued to indicate a crash:

$$r_t^{\text{alt}} = \begin{cases} -1, & 0 < h_t \leq h_{\text{lim}}, \\ -10, & h_t \leq 0, \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

The flight-envelope penalty r_t^{env} prevents the aircraft from entering unsafe or stalled states by penalising excessive angles or overloads that violate the flight envelope:

$$r_t^{\text{env}} = \begin{cases} -1, & |\alpha_t| > \alpha_{\text{lim}} \text{ or } |\beta_t| > \beta_{\text{lim}} \text{ or } v_t > v_{\text{lim}}, \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

In addition, inverted flight frequently occurs in unconstrained manoeuvres but is undesirable for normal control. To mitigate this, we introduce an attitude penalty term r_t^{att} that penalises roll angles exceeding a specified bound (e.g., $\pm 90^\circ$):

$$r_t^{\text{att}} = \begin{cases} \cos(\gamma_t) & \text{if } |\gamma_t| > \pi/2 \\ 0 & \text{other} \end{cases} \quad (8)$$

Under normal manoeuvring conditions, roll angles remain within this bound and incur no penalty.

Together, in contrast to geometric advantage-based reward engineering at the tactical level, the lower-level reward is constructed purely from tracking errors in altitude and the commanded pitch, heading and speed, together with basic safety constraints on the flight envelope and attitude, and does not encode any additional angle or distance-based tactical heuristics.

4. R2SP framework

4.1. Overall architecture of R2SP

Building on the MDP formulation in Sec. 3, we now present the proposed homotopy-guided reinforcement learning framework for WVR air combat, denoted as R2SP (Reward Relabelling and Sub-task Progressing), whose overall architecture is illustrated in Fig. 1. The key idea is to treat training for 1-vs-1 WVR engagements as a *learning-path design problem*: instead of learning directly under extremely sparse combat rewards and fully coupled low-level dynamics, R2SP explicitly reshapes both the temporal distribution of sparse combat-event rewards and the low-level control objective along a coupled-control homotopy. This yields smoother optimisation paths for both tactical decision-making and flight manoeuvring.

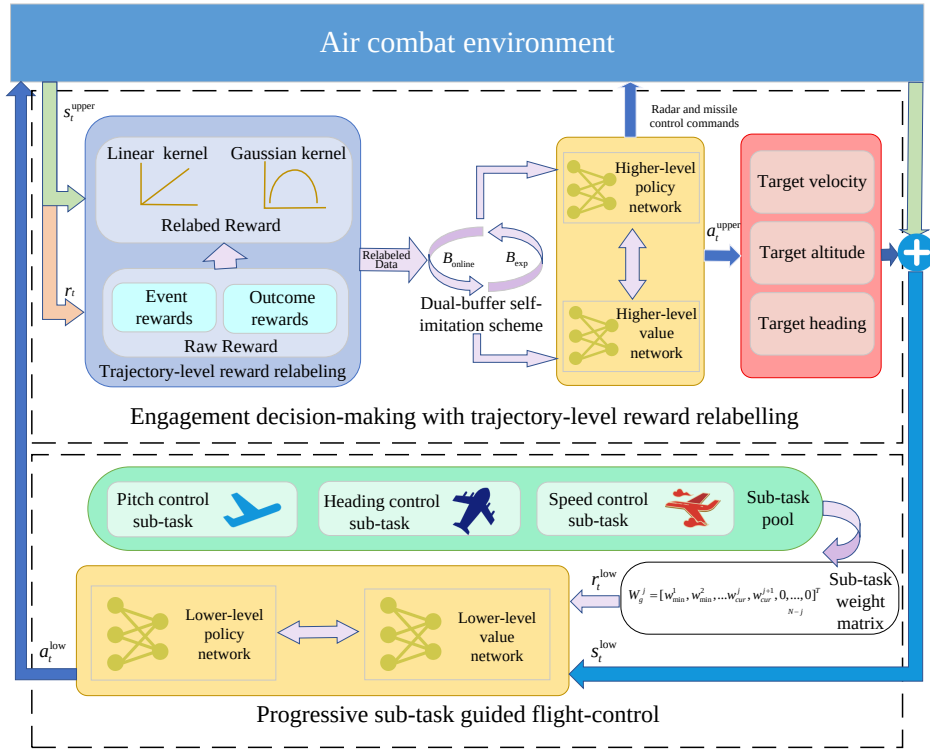


Figure 1: R2SP algorithm architecture.

At the tactical level, R2SP retains the sparse event- and outcome-based environment rewards defined in Sec. 3.2.3 as the only source of air-combat feedback,

but introduces a *trajectory-level reward relabelling* mechanism that redistributes these rewards over time using causal kernels. The relabelled per-step rewards are determined solely by the timing and type of environment events, without introducing additional angle or distance-based geometric advantage terms. This preserves the semantics of the original sparse rewards while providing a denser and more causally aligned training signal, and we further combine this with a dual-buffer self-imitation scheme to better exploit high-return trajectories.

At the execution level, R2SP recognises that the longitudinal (pitch), lateral-directional (heading) and speed channels exhibit heterogeneous dynamics and time scales. Rather than training a monolithic 6-DoF controller from scratch, we design a *progressive sub-task guided flight control* scheme. The overall controller is decomposed into three sub-tasks (pitch, heading and speed control), each associated with a dedicated reward component and weight. A performance-triggered scheduling-and-rollback mechanism gradually morphs the low-level reward from simple to fully coupled control, implementing a principled homotopy over reward functions.

Fig. 1 summarises the overall architecture. The higher-level module observes compact situation states (ownship and opponent kinematics, radar and missile information) and outputs target flight states together with missile-launch commands. These target states are passed to the lower-level controller, which issues continuous elevator, aileron, rudder and throttle commands to the 6-DoF aircraft model. During training, trajectories from the higher-level module are relabelled at the trajectory level and stored in a dual replay buffer, while the lower-level controller is trained under the progressive sub-task reward. In the following, we first detail the trajectory-level reward relabelling for engagement decision-making, and then describe the progressive sub-task guided flight-control learning scheme.

4.2. Engagement decision-making with trajectory-level reward relabelling

The higher-level air-combat decision-making module in R2SP is responsible for generating target flight states and missile-launch commands under the sparse, delayed and adversarial rewards defined in Sec. 3.2. In 1-vs-1 WVR engagements, decisive events such as kill or loss often occur only after long manoeuvre sequences, which makes it difficult for standard off-policy algorithms to assign credit to intermediate tactical actions (Qian et al., 2024).

Conventional approaches typically inject dense geometric rewards (e.g., angle or distance advantages) into the environment to compensate for sparsity, but such hand-engineered shaping terms inevitably encode strong heuristic priors about “good” geometries and may suppress tactically novel solutions. In contrast, R2SP

keeps the environment reward purely event-based and instead modifies its temporal *distribution* along the trajectory. Concretely, each critical combat event is treated as a node whose reward is redistributed backwards over a local time window using a linear or Gaussian kernel. This produces per-step rewards that remain anchored to the original events but provide much richer guidance for temporal credit assignment. From a homotopy perspective, varying the shape and support of these kernels defines a continuous family of reward functions that interpolate between the original sparse objective and its temporally smoothed counterpart, so that approaching a key event generates a gradually increasing reward signal that guides the policy towards tactically promising manoeuvres. The pseudocode is presented in Appendix F.1

4.2.1. Trajectory-level reward relabelling

Let r_t denote the original sparse reward of the higher-level air-combat MDP at time step t . Non-zero rewards occur only at critical events (radar lock-on, missile launch, missile evasion, confirmed kill or loss), so that

$$r_t = \sum_{i=1}^N \mathbb{I}\{t = \tau_i\} R_i, \quad (9)$$

where N is the number of critical events, τ_i is the time step at which the i -th event occurs, R_i is the corresponding event or outcome reward, and $\mathbb{I}\{\cdot\}$ is the indicator function. Directly training on (9) causes strong temporal credit assignment difficulties.

To alleviate this, R2SP introduces a trajectory-level reward relabelling mechanism. For each critical event i , we define a temporal window $W_i = [t_i^{\text{beg}}, t_i^{\text{end}}]$ and a non-negative kernel $k_i(t)$ supported on W_i . The relabelled per-step reward is then

$$\tilde{r}_t(s_t, a_t) = \sum_{i=1}^N k_i(t) R_i, \quad (10)$$

which redistributes the event reward R_i over its local window W_i without introducing any additional domain-specific potentials.

Linear reward relabelling. For terminal or long-horizon outcome rewards, a simple yet effective choice is a linear (uniform) kernel that spreads the reward evenly across a finite interval:

$$k_i(t) = \begin{cases} \frac{1}{t_i^{\text{end}} - t_i^{\text{beg}}}, & t_i^{\text{beg}} \leq t \leq t_i^{\text{end}}, \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

This corresponds to viewing all time steps within W_i as contributing equally to the eventual outcome associated with event i .

For *outcome rewards*, which reflect the final result of the engagement and are influenced by the entire action sequence, we adopt the linear kernel in (11) with $t^{\text{beg}} = 0$. In WVR air combat, the missiles considered are fire-and-forget infrared missiles. After missile launch, the agent’s subsequent manoeuvres have limited influence on the kill outcome. Therefore, the end time of the relabelling window is set to the missile launch time $t^{\text{end}} = T^{\text{missile}}$.

Nonlinear reward relabelling. In many WVR scenarios, actions taken shortly before a critical event (such as missile launch or a successful evasion) influence the outcome much more strongly than actions far in the past. To encode this causal asymmetry, we adopt a truncated Gaussian kernel

$$k_i(t) = \begin{cases} \frac{4}{\sqrt{2\pi} (t_i^{\text{end}} - t_i^{\text{beg}})} \exp\left(-\frac{2(t - t_i^{\text{end}})^2}{(t_i^{\text{end}} - t_i^{\text{beg}})^2}\right), & t_i^{\text{beg}} \leq t \leq t_i^{\text{end}}, \\ 0, & \text{otherwise,} \end{cases} \quad (12)$$

where t_i^{end} is the event time and t_i^{beg} is the start of the causal window. The kernel peaks at t_i^{end} and decays exponentially backwards in time, assigning the largest credit to states and actions immediately preceding the event while still preserving the total reward scale up to a constant factor.

For *event rewards*, we use the nonlinear Gaussian kernel in (12) with $t^{\text{beg}} = 0$ and $t^{\text{end}} = T^{\text{eve}}$, where T^{eve} is the time of the corresponding event. This realises a causal “backward smoothing” of event rewards, giving the highest credit to actions closest in time to the event.

Compared with conventional dense-shaping methods that insert angle or distance-based potentials into the environment, the kernels in (11)–(12) do not create new reward terms: they only *redistribute* existing event and outcome rewards in time. This is crucial for maintaining the sparse-reward semantics of the combat MDP while enabling more informative training signals.

4.2.2. Theoretical properties of trajectory-level reward relabelling

We briefly analyse the theoretical properties of the trajectory-level reward relabelling scheme in the discounted setting, focusing on (i) approximate invariance of the discounted return and (ii) its impact on temporal credit assignment.

Let G_0 and $\tilde{G}_0$ denote the discounted returns under the original and relabelled rewards, respectively,

$$G_0 = \sum_{t=0}^{T-1} \gamma^t r_t, \quad \tilde{G}_0 = \sum_{t=0}^{T-1} \gamma^t \tilde{r}_t, \quad (13)$$

where $\gamma \in (0, 1)$ is the discount factor. Using (9) and (10), their difference can be written as

$$\begin{aligned} \tilde{G}_0 - G_0 &= \sum_{t=0}^{T-1} \gamma^t (\tilde{r}_t - r_t) \\ &= \sum_{t=0}^{T-1} \gamma^t \left(\sum_{i=1}^N k_i(t) R_i - \sum_{i=1}^N \mathbb{I}\{t = \tau_i\} R_i \right) \end{aligned} \quad (14)$$

Assume that each kernel $k_i(t)$ is non-negative, supported on a window $W_i = [t_i^{beg}, t_i^{end}]$ of length at most L , and approximately normalised such that $\sum_{t \in W_i} k_i(t) \approx 1$, and that $|R_i| \leq R_{\max}$. Within the window, t and τ_i differ by at most L , so that $|\gamma^t - \gamma^{\tau_i}| \leq \gamma^{\tau_i} (1 - \gamma^L)$. A coarse bound on the difference is then

$$|\tilde{G}_0 - G_0| \lesssim N R_{\max} (1 - \gamma^L). \quad (15)$$

In the practically relevant regime where the relabelling windows are short (tens of steps) and γ is close to one (e.g., 0.99), the right hand side is small. Thus, trajectory-level reward relabelling can be viewed as introducing a bounded and controllable perturbation to the discounted objective $J_\gamma(\pi)$, while preserving the undiscounted return exactly. This justifies *interpreting the scheme as a form of approximately invariant reward shaping that smooths the temporal distribution of rewards without substantially altering the optimisation target*.

Next, consider the discounted return from an intermediate time step t ,

$$G_t = \sum_{u=t}^{T-1} \gamma^{u-t} r_u, \quad \tilde{G}_t = \sum_{u=t}^{T-1} \gamma^{u-t} \tilde{r}_u, \quad (16)$$

and the corresponding action-value functions

$$Q^\pi(s_t, a_t) = \mathbb{E}_\pi[G_t | s_t, a_t], \quad \tilde{Q}^\pi(s_t, a_t) = \mathbb{E}_\pi[\tilde{G}_t | s_t, a_t]. \quad (17)$$

For a single trajectory, the difference in returns from time t is

$$\tilde{G}_t - G_t = \sum_{i=1}^N R_i \left(\sum_{u=t}^{T-1} \gamma^{u-t} k_i(u) - \gamma^{\tau_i-t} \mathbb{I}\{t \leq \tau_i\} \right). \quad (18)$$

Equation (18) highlights the effect of reward relabelling on temporal credit assignment:

- For time steps t outside all kernel windows, the inner sum vanishes and $\tilde{G}_t = G_t$; value estimates remain unchanged in those regions.
- For time steps t within the window of some event i , the original contribution of R_i is a single discounted spike $\gamma^{\tau_i-t} R_i$, whereas after relabelling this is replaced by a discounted convolution of R_i with $k_i(u)$ over the window. *This effectively turns a sharp peak in the value landscape into a smoother uphill slope along the approach to the event.*

As a consequence, states and actions lying on trajectories that actually lead to key tactical events (such as establishing and maintaining a firing solution) receive consistent positive contributions to their value estimates over contiguous time intervals, instead of only at the terminal event time. This increases the advantage margin between actions that steer the agent towards promising tactical opportunities and those that do not, and yields more locally observable gradients for the learner. Empirically, this improves escape from conservative sub-optimal behaviours (e.g., overly passive manoeuvring) and promotes convergence to policies that more effectively exploit favourable engagement opportunities.

Additionally, from the perspective of homotopy-guided learning, the kernel parameters (such as the window length and the spread of the Gaussian kernel) define a continuum between the original sparse reward r_t and the fully relabelled reward $\tilde{r}_t$. One can view $\tilde{r}_t$ as the endpoint of a conceptual reward homotopy that gradually redistributes each event or outcome reward R_i from a single time step to a small neighbourhood along the trajectory, while leaving the undiscounted return unchanged and perturbing the discounted return only within the bounded range discussed above.

4.2.3. Algorithmic realisation with dual-buffer self-imitation

To fully exploit the relabelled rewards, the higher-level module of R2SP adopts a dual-buffer self-imitation structure that separates high-return trajectories from ordinary online experience while maintaining sufficient exploration. Two kinds of buffers are maintained:

- **Online policy buffer** $\mathcal{B}_{\text{online}}$, which stores trajectories with relatively low or moderate total returns. The sparse environment rewards in two trajectories are relabelled using the mechanism in Sec. 4.2.

- **Heuristic strategy buffer $\mathcal{B}_{\text{exp}}$** , which stores trajectories with relatively high total returns and serves as a source of self-imitation data. Rewards in these trajectories are also relabelled so that the agent can repeatedly learn from its own high-return behaviour.

After each episode, we compute the average per-step reward $r_{\text{avg}}^{\text{ep}}$ of the trajectory and compare it to the average per-step reward $r_{\text{avg}}^{\text{exp}}$ of trajectories already in $\mathcal{B}_{\text{exp}}$:

$$r_{\text{avg}}^{\text{ep}} > \alpha r_{\text{avg}}^{\text{exp}}, \quad (19)$$

where α is a scaling factor. If (19) holds, the trajectory is added to $\mathcal{B}_{\text{exp}}$; otherwise, it is added to $\mathcal{B}_{\text{online}}$. In this way, $\mathcal{B}_{\text{exp}}$ acts as a high-quality experience buffer, whereas $\mathcal{B}_{\text{online}}$ maintains a diverse set of on-policy samples.

During training, mini-batches are sampled from $\mathcal{B}_{\text{online}}$ and $\mathcal{B}_{\text{exp}}$ with a dynamic sampling ratio λ_t that evolves according to

$$\lambda_t = \begin{cases} 0, & t < t^{\text{beg}}, \\ \frac{\lambda^{\text{end}} - \lambda^{\text{beg}}}{t^{\text{end}} - t^{\text{beg}}}(t - t^{\text{beg}}) + \lambda^{\text{beg}}, & t^{\text{beg}} \leq t \leq t^{\text{end}}, \\ \lambda^{\text{end}}, & t > t^{\text{end}}, \end{cases} \quad (20)$$

where λ^{beg} and λ^{end} are the initial and final sampling ratios, respectively. A fraction λ_t of each batch is drawn from $\mathcal{B}_{\text{exp}}$ and the remainder from $\mathcal{B}_{\text{online}}$, so that the influence of heuristic trajectories decreases gradually as learning progresses.

The higher-level decision-making module can be instantiated with any off-policy DRL algorithm; in this study we employ the widely used SAC algorithm. The overall training procedure consists of:

1. **Pre-training.** We first populate the buffers with a small set of suboptimal demonstration trajectories, e.g., those generated by the rule-based tactics described in Sec. 5.1.2. Each trajectory is relabelled and then assigned to $\mathcal{B}_{\text{exp}}$ or $\mathcal{B}_{\text{online}}$ according to (19). The SAC agent is pre-trained on these trajectories to obtain a reasonable initial policy.
2. **Online training.** The agent subsequently interacts with the environment using its current policy. After each episode, the trajectory is relabelled, classified and inserted into $\mathcal{B}_{\text{exp}}$ or $\mathcal{B}_{\text{online}}$. The sampling ratio λ_t follows (20), so that high-return experience gradually shifts from external demonstrations to the agent’s own trajectories.

4.3. Progressive sub-task guided flight-control learning

While the higher-level module learns tactical policies under relabelled sparse rewards, the aircraft still requires an inner-loop controller to execute three-dimensional manoeuvres and track the target flight states issued by the tactical policy. As discussed in Sec. 3.3, jointly learning a monolithic controller for coupled pitch, heading and speed channels in a high-fidelity 6-DoF model is challenging and often leads to unstable training or suboptimal convergence.

To better exploit the heterogeneous characteristics of the control channels, R2SP decomposes the low-level tracking task into three sub-tasks—pitch control, heading control and speed control—and organises training via a *progressive sub-task guided learning* mechanism. Intuitively, the controller is first required to master simple manoeuvres (e.g., stabilising pitch), and only after reliable performance is achieved is it gradually assigned more complex objectives (e.g., coupled heading and speed control). If performance on a newly activated sub-task deteriorates, the algorithm temporarily backtracks to earlier sub-tasks. This implements a curriculum over reward functions that guides the optimiser along a favourable path in the control-policy space.

4.3.1. Algorithm design

Let $s_t \in \mathcal{S}$ and $a_t \in \mathcal{A}$ denote the state and action of the lower-level controller. We define a vector of sub-task rewards

$$\mathbf{R}(t) = [R_1(t), R_2(t), \dots, R_K(t)]^\top, \quad (21)$$

where each component $R_k(t)$ evaluates the performance of one sub-task. The sub-task index j ($1 \leq j \leq N$) indicates the current stage of training, and the active sub-task is denoted by $s_j^g \in \mathcal{E}_g$. The instantaneous low-level reward at stage j is given by

$$r_t^{\text{sta}}(s_t, a_t, s_j^g) = \mathbf{W}_g^j \mathbf{R}(t)^T \quad (22)$$

where $\mathbf{W}_g^j$ is a row vector of non-negative task weights.

At stage j , the weight vector takes the form

$$\mathbf{W}_g^j = \left[w_{\min}^1, w_{\min}^2, \dots, w_{\min}^j, w_{\min}^{j+1}, \underbrace{0, \dots, 0}_{N-j-1} \right] \quad (23)$$

where $w_{\min}^j$ and $w_{\min}^{j+1}$ are weights that are currently being dynamically adjusted, $w_{\min}^1$ to $w_{\min}^k$ correspond to the minimum reward weights for the first k sub-tasks.

To formalise the gradual activation of sub-tasks, we introduce an achievement counter n_{cur} and a threshold N_{count} . During the training of stage j , the weights for sub-tasks j and $j + 1$ evolve according to

$$w_j^{\text{cur}} = w_j^{\text{max}} - \frac{w_j^{\text{max}} - w_j^{\text{min}}}{N_{\text{count}}} n_{\text{cur}}, \quad n_{\text{cur}} \leq N_{\text{count}}, \quad (24)$$

$$w_{j+1}^{\text{cur}} = \frac{w_{j+1}^{\text{max}}}{N_{\text{count}}} n_{\text{cur}}, \quad n_{\text{cur}} \leq N_{\text{count}}. \quad (25)$$

Whenever a valid learning iteration is completed with satisfactory performance, the counter is updated as $n_{\text{cur}} \leftarrow n_{\text{cur}} + 1$. When the current sub-task meets its achievement criterion for N_{count} consecutive steps, sub-task j is considered mastered, and the algorithm transitions to the next stage $j+1$. Importantly, the weights $w_{\text{min}}^1, \dots, w_{\text{cur}}^j$ remain non-zero so that previously acquired skills are not forgotten.

To guard against failures on newly activated sub-tasks, R2SP includes an adaptive backtracking mechanism. Let R_n^j denote the total reward of sub-task j for the n -th trajectory and τ_{thr}^j be the minimum acceptable reward threshold. If

$$R_n^j \leq \tau_{\text{thr}}^j \quad (26)$$

holds for N_{thr} consecutive trajectories, the algorithm reverts to stage j from stage $j+1$ and temporarily suppresses the weight of the problematic sub-task. This allows the controller to consolidate earlier skills before attempting the more difficult objective again.

The sub-task space is defined in Eq. (27)

$$E_{\text{g}} = \cup_{k=1}^3 s_{\text{g}}^k \quad (k = 1, 2, 3) \quad (27)$$

where $k = 1, k = 2$ and $k = 3$ correspond to pitch, heading and speed control sub-tasks, respectively. The reward for each sub-task is computed according to Eq. (28)

$$\mathbf{R}(t) = [e^{-|18\Delta\theta_t/\pi|}, e^{-|18\Delta\varphi_t/\pi|}, e^{-|20\Delta M_t|}], \quad (28)$$

where $\Delta\theta_t, \Delta\varphi_t$ and ΔM_t denote the tracking errors in pitch angle, heading angle and Mach number, respectively. These quantities provide clear and physically interpretable measures of sub-task performance.

The lower-level controller can be trained with any off-policy DRL algorithm. In this work we employ the TD3 algorithm, which is well-suited to continuous control. The pseudocode is presented in Appendix F.2

4.3.2. Theoretical perspective: homotopy effects

The progressive sub-task guided control scheme can be interpreted as a *reward homotopy* for the low-level controller, which provides a useful theoretical perspective on its stabilising effect.

Recall that the stage- j low-level reward is

$$r_t^{\text{sta},(j)} = \mathbf{W}_g^j \mathbf{R}(t)^\top, \quad (29)$$

while the final desired objective corresponds to a target weight vector $\mathbf{W}_g^*$ and reward

$$r_t^{\text{sta},*} = \mathbf{W}_g^* \mathbf{R}(t)^\top. \quad (30)$$

Instead of optimising $r_t^{\text{sta},*}$ from scratch, progressive sub-task control constructs a sequence of intermediate MDPs whose rewards are determined by $\{\mathbf{W}_g^j\}_{j=1}^J$, where $\mathbf{W}_g^1$ emphasises only the easiest sub-task and $\mathbf{W}_g^J$ is close to the final weight configuration $\mathbf{W}_g^*$.

Conceptually, this can be viewed as sampling a continuous homotopy path in the space of weight vectors:

$$\mathbf{W}_g(\lambda) = (1 - \lambda)\mathbf{W}_g^1 + \lambda\mathbf{W}_g^*, \quad \lambda \in [0, 1], \quad (31)$$

with the corresponding family of rewards

$$r_t^{\text{sta}}(\lambda) = \mathbf{W}_g(\lambda) \mathbf{R}(t)^\top. \quad (32)$$

Here $\lambda = 0$ recovers the simplest sub-task and $\lambda = 1$ the full low-level objective. Under mild regularity assumptions (e.g., continuity of the optimal value function and policy with respect to λ), slowly increasing λ defines a continuation path in policy space: a gradient-based learning algorithm that is allowed to approximately converge for a given λ and then warm-started for a slightly larger λ' tends to track a continuous trajectory of near-optimal policies from the simple task to the full task.

In R2SP, this homotopy idea is implemented in a discrete, performance-driven manner:

- Each training stage j fixes a weight configuration $\mathbf{W}_g^j$ that approximates $\mathbf{W}_g(\lambda_j)$ for some λ_j .
- Stage transitions are triggered only when the current sub-task meets its achievement criterion over a number of steps (via the achievement counter and threshold), ensuring that the policy has reached a reasonably good local optimum for the simpler objective before it is made more difficult.

- The rollback mechanism downgrades from stage $j+1$ to stage j if the new reward configuration causes persistent performance degradation (condition (26)). This prevents the optimiser from being driven into poor regions of the landscape when the objective is changed too aggressively.

Viewed from this homotopy perspective, the progressive sub-task scheme realises a curriculum over reward functions that guides the low-level controller along a “good” solution manifold, rather than forcing it to search the full high-dimensional objective from scratch. This provides a theoretical explanation for the empirical observations in Sec. 5: compared with joint training on the full low-level objective, the progressive scheme exhibits smoother learning curves, improved stability, and better final tracking and engagement performance.

5. Experimental results

5.1. Experimental design

In this section, we empirically evaluate the proposed homotopy-guided RL framework R2SP from four complementary perspectives that are aligned with the two core innovations in Sec. 4: (i) trajectory-level reward relabelling for sparse-reward tactical decision-making, and (ii) progressive sub-task guided flight-control learning. Concretely, we aim to answer the following questions:

- **RQ1 (Overall effectiveness Sec. 5.2)).** Does R2SP, which couples trajectory-level reward relabelling with progressive sub-task control, achieve better training efficiency and combat performance than state-of-the-art DRL baselines and rule-based tactics?
- **RQ2 (Low-level control homotopy Sec. 5.3).** Does the progressive sub-task guided scheme improve flight-control learning stability and tracking accuracy?
- **RQ3 (High-level reward relabelling Sec. 5.4).** How much does trajectory-level reward relabelling contribute to learning under sparse event and outcome-based rewards, compared with directly optimising on unprocessed sparse rewards?
- **RQ4 (Iterative evolution Sec. 5.5).** Under self-play training, can R2SP exploit the reward–control homotopy to yield iteratively evolving, tactically meaningful manoeuvres and convergent multi-generation policies?

5.1.1. *Experimental objectives and evaluation protocol*

We evaluate the proposed R2SP framework in a high-fidelity 6-DoF WVR air-combat simulator, and organise the experiments according to the four research questions above. Together, these experiments provide a systematic evaluation of how the two key mechanisms — reward relabelling and progressive sub-task control — translate into improved sample efficiency, control quality and tactical performance at the system level.

5.1.2. *Simulation environment and scenario configuration*

All experiments are conducted in a WVR simulation environment based on the JSBSim F-16 aircraft model. The native JSBSim environment provides a 6-DoF aircraft model and basic flight dynamics. To support WVR engagements, we augment JSBSim with custom modules for a fire-control radar and infrared air-to-air missiles. The kinematic, guidance and detection models of the radar and missiles are given in Appendix C and Appendix D. Algorithmic hyperparameters and training configurations for R2SP and all baselines are summarised in Appendix E

Scenario configuration. The initial attitudes of the red-side and blue-side aircraft are randomised. The red-side aircraft, controlled by the learning algorithm (R2SP or a baseline DRL method), is initialised with altitude uniformly sampled from $[3, 8]$ km. The blue-side aircraft is positioned such that the initial distance between the two aircraft lies within $[10, 15]$ km and the altitude difference is less than 2 km.

Opponent policy and demonstration data. During training, the blue-side aircraft follows a rule-based policy that commands it to orbit around its initial position at a constant speed of Mach 0.6, thereby maintaining a WVR engagement geometry and providing a dense stream of tactically meaningful interactions. This rule-based policy is also used offline to generate a small set of suboptimal demonstration trajectories for the higher-level agent. Specifically, we collect about 100 engagement episodes under the above scenario configuration, relabel the sparse rewards according to Sec. 4.2, and store the resulting trajectories in the heuristic strategy buffer to initialise R2SP in the pre-training phase. Details are provided in Appendix G.

Simulation parameters. The radar detection radius for both sides is 5 km, with a field of view of 30° . The decision interval for high-level air combat planning is 0.4 s, while the simulation time step is 0.01 s. Each episode contains at most 1 200 high-level decision steps and terminates when either aircraft is shot down (hit by a missile) or crashes (altitude ≤ 0).

5.2. Overall performance against baselines and rule-based opponents

We first address RQ1 by evaluating the overall effectiveness of R2SP as a complete homotopy-guided framework. All methods use the same 6-DoF environment, sparse event-based tactical rewards and training protocol; the only difference lies in whether they exploit the proposed trajectory-level reward relabelling and progressive sub-task flight control.

5.2.1. Reward-learning curve comparison

The baselines include representative off-policy and on-policy continuous-control algorithms: SAC (Yoo et al., 2022), DSAC (Duan et al., 2025), TD3, REDQ (Chen et al., 2021), TQC (Kuznetsov et al., 2020), PPO and DDPG. All agents observe the same high-level state, share the same action parameterisation and are trained with identical network architectures and optimisation settings.

Figure 2 shows the evolution of training rewards over episodes. R2SP exhibits a clear advantage over all baselines: the average return increases rapidly and essentially converges within about 1 000 episodes, without noticeable regression as training proceeds, indicating stable optimisation behaviour. Among the baselines, SAC achieves the best performance, with relatively smooth but slower reward growth. DSAC, TD3 and REDQ also achieve non-trivial returns but their curves display larger oscillations. TQC, PPO and DDPG fail to obtain high rewards in this sparse and adversarial WVR environment, reflecting their limited ability to exploit rare but high-impact events.

It is important to note that R2SP does *not* use any handcrafted geometric shaping terms in the tactical reward. The improvement in convergence speed and final return therefore stems from (i) the trajectory-level reward relabelling scheme, which reshapes sparse rewards into more informative per-step signals, and (ii) the progressive sub-task guided low-level control, which stabilises the execution of commanded tactical manoeuvres. The learning curves thus provide direct empirical evidence that the reward-control homotopy leads to a significantly easier optimisation problem than standard DRL training under the original sparse rewards.

5.2.2. Win-rate evaluation

To further quantify combat effectiveness, we select four strong DRL baselines (SAC, DSAC, TD3 and REDQ) based on their training performance and compare them with R2SP in direct WVR engagements. Each trained agent engages in 1 000 encounters against the R2SP-controlled red-side aircraft under the same scenario configuration, and the outcomes (win, loss, tie) are recorded from the perspective

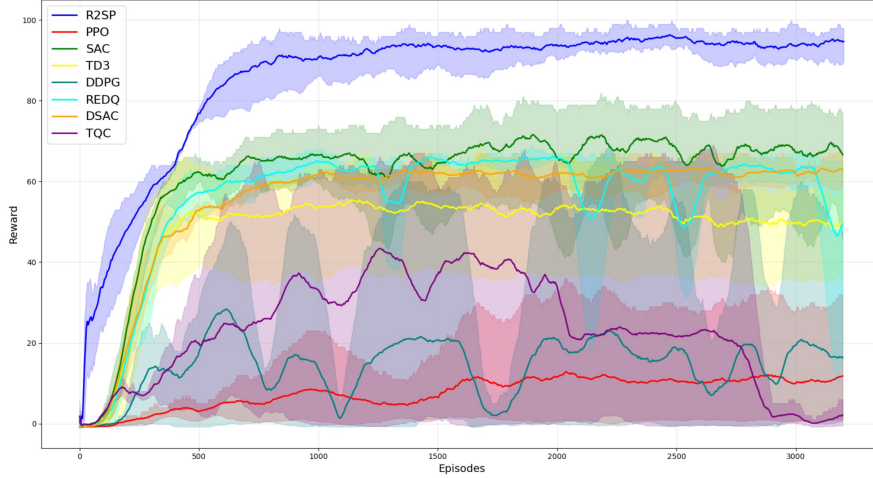


Figure 2: R2SP vs. SOTA algorithms.

of R2SP. In addition, we evaluate R2SP against the rule-based air-combat policy that generates the demonstration trajectories.

The results, summarised in Figure 3, show that R2SP achieves an overall win rate of about 73.5% against all competitors (excluding ties), while the probability of being at a disadvantage remains below 18.1%. This indicates that R2SP not only learns a robust policy against the rule-based opponent that provided initial demonstrations, but also maintains clear superiority over strong DRL opponents trained under identical conditions.

Combining the reward-learning curves and win-rate statistics, we conclude that the proposed homotopy-guided framework substantially improves both training efficiency and operational effectiveness. From an algorithmic perspective, these results validate that jointly reshaping the sparse tactical reward over time and progressively constructing the low-level control objective is crucial for turning the original challenging WVR task into one that can be solved reliably by off-policy DRL.

5.3. Performance of sub-task guided flight-control learning

This subsection evaluates the lower-level flight-control module of R2SP from two perspectives: 1) *Sub-task convergence* under the progressive sub-task guidance mechanism; 2) *Control precision and trajectory quality* compared to baseline algorithms and a curriculum learning-based controller.

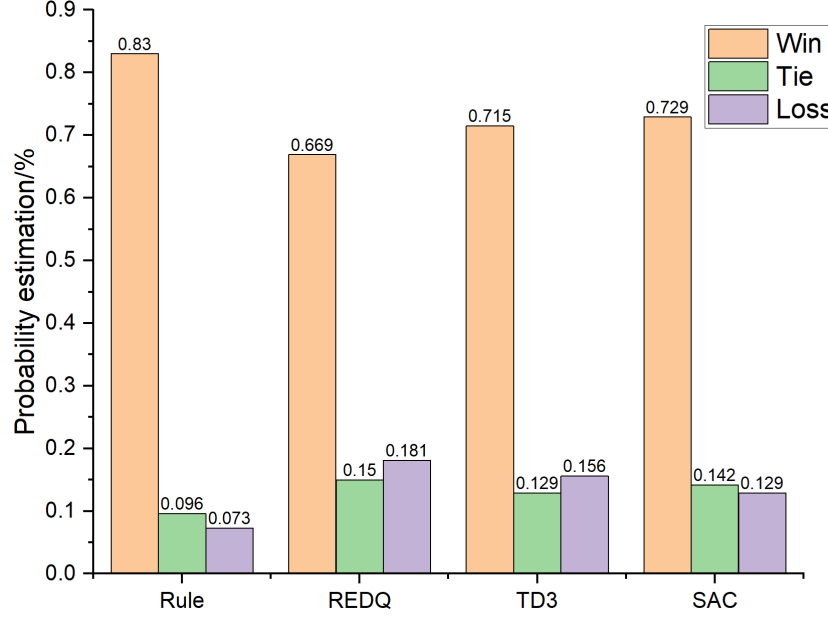


Figure 3: R2SP vs. rule-based and SOTA algorithms.

5.3.1. Sub-task completion

At the start of each training episode, a sequence of sub-tasks (climb/dive, turning and acceleration/deceleration) is randomly generated. Each episode consists of 1 200 simulation steps. The R2SP controller is trained using the progressive sub-task mechanism, and sub-task completion statistics are recorded over time.

Figure 4 reports the completion rates of the three sub-tasks across training episodes. Sub-task 0 (climb/dive) converges in fewer than 500 episodes, after which the agent reliably maintains the commanded altitude profile. Sub-task 1 (turning) requires more training due to stronger coupling between roll and yaw dynamics, while Sub-task 2 (acceleration/deceleration) is the most challenging because it involves coordinated use of throttle and attitude. Despite this increasing difficulty, the completion curves remain monotonic and do not exhibit catastrophic drops, indicating that the progressive scheme effectively balances skill acquisition and retention.

These results empirically support the homotopy interpretation in Sec. 4.3.2: by gradually activating sub-tasks and allowing rollback when performance deteriorates, the controller follows a smooth path in policy space from simple to complex manoeuvres, instead of attempting to solve the full high-dimensional control

objective from scratch.

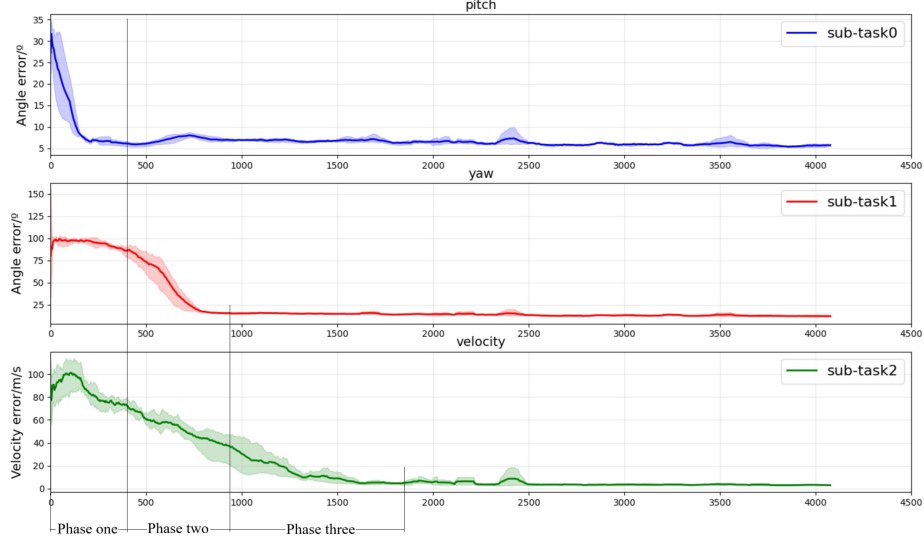


Figure 4: Comparison of subtask completion levels.

5.3.2. Reward comparison

Because the R2SP controller employs stage-dependent sub-task weights in its reward, a fair numerical comparison with baselines requires a normalised evaluation metric. For this purpose, we reset the sub-task weights $\mathbf{W}_g^j$ to a fixed configuration when computing *evaluation* rewards, so that both R2SP and baseline controllers are judged under the same scalar objective. The unified reward is primarily determined by the discrepancy between the current flight state and the target state, without progressive weighting.

Figure 5 presents the reward-learning curves of R2SP, SAC, TD3 and DDPG under this unified metric. R2SP achieves substantially higher rewards than SAC and TD3 and converges smoothly to a high-performance regime. SAC and TD3, which attempt to learn all control channels simultaneously without explicit sub-task guidance, tend to converge to suboptimal solutions and exhibit larger oscillations in their reward curves. DDPG performs worst due to its limited exploration capability and sensitivity to high-dimensional continuous control.

Together with the sub-task completion statistics, these results demonstrate that the progressive sub-task guided design is essential for efficient and stable learning of precise flight control in the high-fidelity 6-DoF environment.

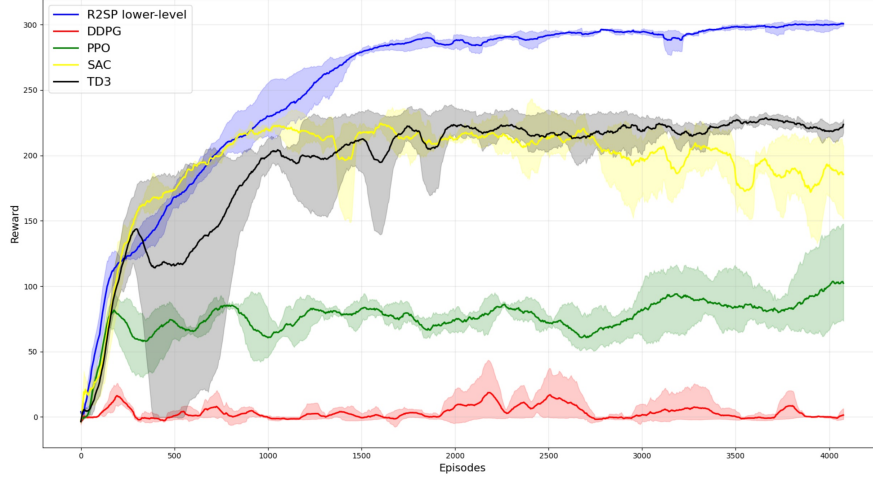


Figure 5: Reward comparison results.

5.3.3. Flight trajectory comparison

To further assess control precision, we design a multi-target waypoint-tracking task. The aircraft starts from a given waypoint (e.g., $(120^\circ\text{E}, 25^\circ\text{N})$) at an altitude of 4 000 m. Five target waypoints are generated sequentially within a sphere of radius 40 km centred at the initial position and projected onto the altitude plane at 4 000 m. The aircraft is required to pass through all five waypoints in order while minimising tracking errors.

Figure 6 compares the trajectories and altitude profiles produced by R2SP, the monolithic SAC controller and the CL-based controller. The R2SP controller tracks all waypoints accurately, with smooth attitude changes and small overshoots in both lateral and vertical channels. The monolithic SAC controller exhibits noticeable overshoot and oscillations, especially during turning segments. The CL-based controller performs better than SAC but still shows larger heading and altitude deviations than R2SP.

These qualitative observations are consistent with the quantitative reward comparison and further confirm that progressive sub-task guided learning yields more precise and robust flight-control behaviour than monolithic or generic curriculum-based schemes.

5.4. Ablation study of trajectory-level reward relabeling

We now address RQ3 by isolating the effect of trajectory-level reward relabelling on higher-level tactical decision-making. R2SP is compared with baseline

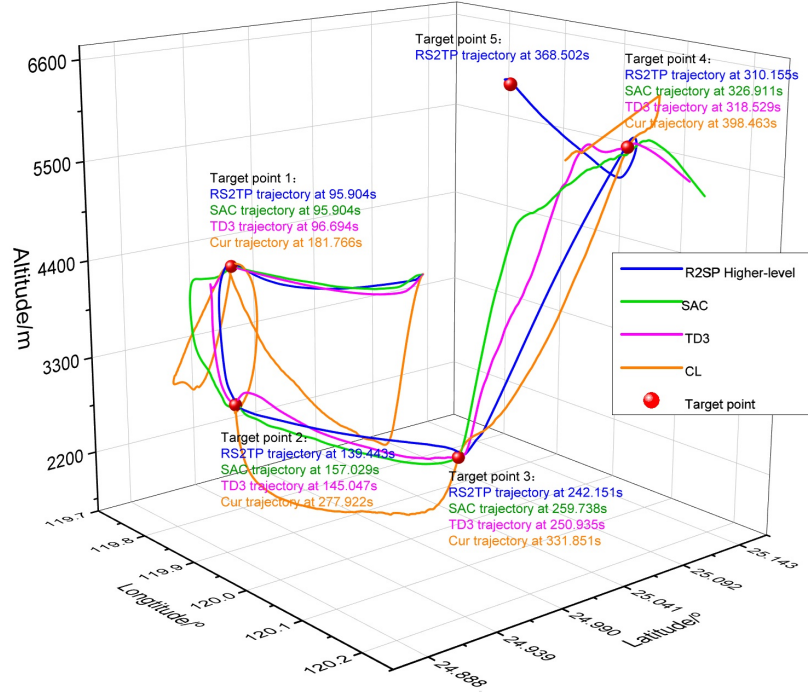


Figure 6: Comparison of multi-target flight trajectory.

DRL algorithms that operate directly on the sparse event and outcome rewards in Sec. 3.2.3, without any temporal relabelling.

Figure 7 presents the reward-learning curves for R2SP and representative baselines such as SAC. R2SP achieves rapid convergence within approximately 1 000 episodes and reaches a significantly higher reward level. By contrast, baselines trained with unprocessed sparse rewards exhibit slow and unstable learning: their reward curves grow gradually and saturate at substantially lower values.

It is worth emphasising that the reward relabelling mechanism does not rely on hand-crafted geometric advantage functions or encoded air-combat rules. Instead, it operates purely at the trajectory level by redistributing environment-provided sparse rewards using linear and nonlinear temporal kernels, while keeping the semantics of event and outcome rewards unchanged. This makes the method conceptually distinct from conventional “advantage-based reward shaping” that injects domain-specific potentials into dense rewards.

Together with the theoretical analysis in Sec. 4.2.2, these ablation results show that trajectory-level reward relabelling provides a principled and effective way to alleviate severe reward sparsity in WVR air combat. By creating smoother

and more causally aligned value landscapes, it allows off-policy DRL algorithms such as SAC to escape conservative local optima and converge to aggressive yet tactically sound engagement strategies.

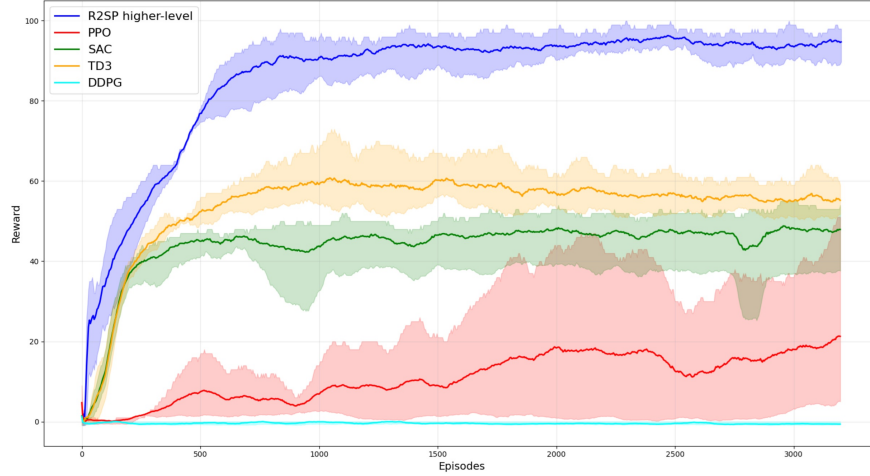


Figure 7: Reward comparison between R2SP and baseline algorithms.

5.5. Analysis of iterative evolution capability of R2SP

Finally, we investigate RQ4: the iterative evolution capability of R2SP under a self-play training regime. The self-play protocol, including opponent sampling, generation update and evaluation rules, is summarised in Appendix B. We conduct two sets of experiments:

- *Win-rate comparison across R2SP generations.* We measure how policy performance improves and stabilises as new generations are trained against a mixture of earlier ones.
- *Cross-algorithm comparison of evolution capability.* We contrast R2SP with several SOTA DRL baselines trained under the same self-play protocol.

5.5.1. Win-rate comparison across policy generations

The converged R2SP model from Sec. 5.2 serves as the first-generation policy in self-play. Subsequent generations are obtained iteratively using the training/evaluation cycle defined in Appendix A. Once a generation converges and satisfies the evaluation criterion, its policy is added to the strategy pool.

Table 2 reports the win-rate matrix among policy generations, where each entry records the win/tie/loss rates of the row strategy against the column strategy. Higher-generation policies consistently achieve higher win rates against earlier generations, and the matrix exhibits a clear dominance pattern rather than cyclic preferences. This indicates that R2SP under self-play evolves towards a stable and increasingly strong strategy instead of overfitting to specific opponents.

Table 2: Win-rate comparison between different R2SP policy generations.

R2SP	Gen 1	Gen 2	Gen 3	Gen 4
Red/Blue	Win /Tie /Loss	Win /Tie /Loss	Win /Tie /Loss	Win /Tie /Loss
Gen 1	0.42/0.18/0.40	0.24/0.16/0.60	0.22/0.15/0.63	0.15/0.15/0.70
Gen 2	0.60/0.16/0.24	0.44/0.15/0.41	0.35/0.20/0.45	0.28/0.16/0.56
Gen 3	0.63/0.15/0.22	0.55/0.20/0.25	0.42/0.17/0.41	0.38/0.16/0.46
Gen 4	0.70/0.15/0.15	0.56/0.16/0.28	0.46/0.16/0.38	0.42/0.17/0.41

5.5.2. Comparison of evolution capability across algorithms

We further compare the evolution capability of R2SP with SAC, DSAC and TD3 under the same self-play protocol. For each algorithm, we record the win rates of the final self-play generation against all earlier generations and against the converged R2SP policy.

As shown in Figure 8, although baseline algorithms can achieve moderate improvements through self-play, their final policies remain clearly inferior to R2SP. In particular, they either exhibit unstable trajectories with frequent crashes or adopt overly conservative manoeuvres that fail to exploit transient firing opportunities. In contrast, R2SP produces tactically meaningful manoeuvres that evolve iteratively across generations, such as actively shaping engagement geometry to obtain stable radar lock and exploiting favourable missile envelopes.

Overall, the self-play experiments demonstrate that the proposed homotopy-guided RL framework not only improves one-shot training performance, but also enables robust iterative evolution in adversarial settings. This further supports the view that reshaping the learning path via reward and control homotopies is a promising direction for sparse-reward, high-dimensional air-combat decision-making.

5.6. Analysis of air combat behaviour

To complement above quantitative evaluations, this section examines whether the WVR air combat strategies learned by R2SP are tactically reasonable and

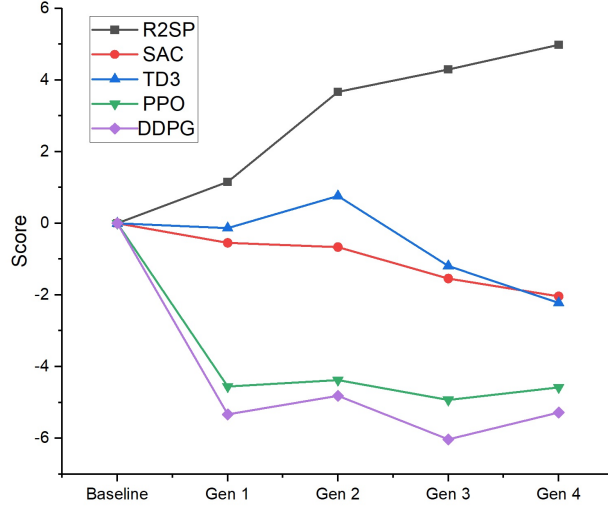


Figure 8: Performance comparison of different algorithms under the self-play architecture.

interpretable from a human pilot’s perspective. In particular, we are interested in whether the agent’s manoeuvres follow classical air combat principles and can compete with, or even surpass, human performance under identical conditions.

5.6.1. Human–AI engagement environment

To this end, we construct a human–AI air combat simulation environment to assess the practical applicability and tactical plausibility of the WVR air combat strategies learned by R2SP.

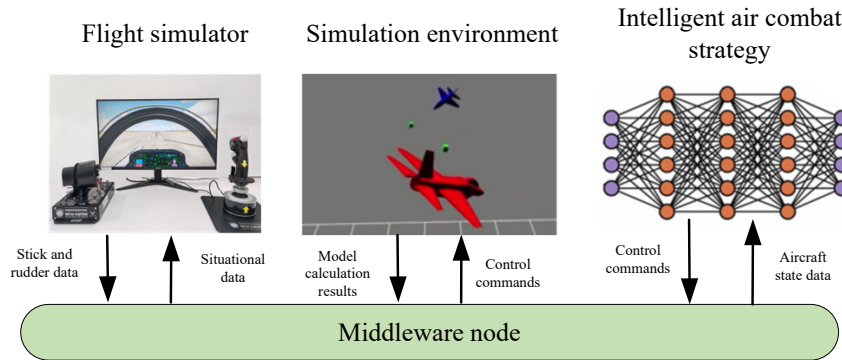


Figure 9: Human–AI air combat simulation environment.

As illustrated in Figure 9, the environment consists of four main components:

1. A *flight simulator* that provides human-in-the-loop control through a throttle quadrant, joystick and rudder pedals, coupled with visual simulation software that presents a cockpit-level out-the-window view.
2. The *intelligent air-combat agent* (R2SP) that outputs high-level target states and missile commands.
3. A high-fidelity *simulation environment* (JSBSim-based WVR air combat environment) in which both the human and AI-controlled aircraft operate.
4. A *middleware node* that serves as a communication bridge, transmitting the aircraft state from the simulation environment to both the human-pilot interface and the R2SP decision-making module, and sending back the corresponding control commands over a local area network (LAN).

In the human–AI engagement, both sides use identical aircraft models, missile configurations and initial conditions to ensure a fair comparison. The human pilot controls one aircraft through the flight simulator, while the R2SP agent directly drives an identical aircraft within the simulation environment via the middleware interface. The R2SP agent runs the converged policy obtained in Sec. 5 without any additional fine-tuning in the human–AI setting. The initial geometry is set to a head-on WVR encounter, and each side aims to achieve a missile kill within a prescribed engagement time. Key kinematic states (such as relative distance and line-of-sight angle), sensor statuses (e.g., radar lock-on) and weapon-employment events are recorded throughout the engagement.

5.6.2. Case study: typical engagement from a disadvantaged initial state

We now present a representative engagement sequence to qualitatively analyse the tactical behaviour of the R2SP agent. In this scenario, the R2SP-controlled aircraft starts from a disadvantaged position relative to the human pilot, for example in a tail-chase or slightly inferior energy state.

Figure 10 shows snapshots of the engagement:

1. The blue-side (human-controlled) aircraft first achieves radar lock-on (Figure 10(a)) and launches an infrared missile (Figure 10(b)).
2. The R2SP agent executes an effective missile-evading manoeuvre, combining hard turns and energy management to break the line-of-sight and kinematic constraints of the incoming missile, successfully evading the threat (Figure 10(c)).



(a) Blue-side aircraft achieves radar lock-on.



(b) Blue-side aircraft launches missile.



(c) Agent successfully evades missile.



(d) Agent executes high yo-yo manoeuvre.



(e) Agent lock-on and launches missile.



(f) Agent successfully hits blue-side aircraft.

Figure 10: Air combat process in a disadvantaged state.

3. After defeating the missile, the agent performs a high yo-yo manoeuvre (Figure 10(d)) to convert altitude advantage into turn-rate and position itself behind the opponent.
4. The R2SP aircraft then achieves radar lock-on and launches its own missile (Figure 10(e)), ultimately scoring a hit on the blue-side aircraft (Figure 10(f)).

The sequence reveals that the WVR air combat strategies learned by R2SP are tactically reasonable and interpretable from a pilot’s perspective. The agent not only reacts appropriately to immediate threats (missile evasion) but also anticipates future engagement geometry (using high yo-yo to gain positional and energy advantage). These behaviours are consistent with classical fighter tactics and are not explicitly encoded in the reward design or heuristic policy, but emerge from the interaction between high-level reward relabelling and low-level sub-task guided control.

6. Conclusion

This paper has presented R2SP, a homotopy-guided reinforcement learning framework for sparse-reward WVR air combat. By jointly reshaping trajectory-level event rewards and low-level control objectives along a reward–control homotopy, R2SP directly addresses sparse-reward credit assignment at the tactical level and tightly coupled flight dynamics at the execution level, going beyond handcrafted geometric advantage functions, heuristic reward shaping and generic curricula that often undermine robustness and transferability across scenarios.

Extensive experiments in a high-fidelity F-16 WVR simulation environment show that, under the same sparse-reward setting, R2SP converges faster and achieves higher returns and win rates than state-of-the-art DRL baselines and rule-based tactics. Component-wise analyses indicate that progressive sub-task control stabilises training and improves trajectory tracking, while trajectory-level reward relabelling enhances learning efficiency and final performance under severe reward sparsity. In multi-generation self-play and human–AI engagements, R2SP policies continue to improve, exhibit pilot-consistent manoeuvres, and can even outperform human pilots, confirming that the proposed mechanisms are effective for realistic sparse-reward WVR air combat.

Overall, R2SP provides a principled homotopy-guided route to tackling sparse-reward adversarial tasks that couple high-level planning with tightly coupled continuous control in realistic air-combat environments. The core ideas of

trajectory-level reward relabelling and progressive sub-task guided control are not tied to a specific aircraft type or engagement geometry and can, in principle, be extended to other adversarial domains with sparse feedback and coupled dynamics, such as beyond-visual-range air combat, multi-agent cooperative engagements and autonomous manoeuvring in contested environments. Future work will explore these extensions and investigate the integration of higher-level mission planning and multi-agent coordination on top of the R2SP framework.

7. CRediT authorship contribution statement

8. Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

9. Acknowledgment

This paper is supported by the National Natural Science Foundation of China (Grant No. 62306329), the Hunan Provincial Natural Science Foundation of China (Grant No.2024JJ6478, No. 2023JJ40676), and the China Association for Science and Technology (CAST) Youth Talent Supporting Program (Grant No. 2024-JCJQ-QT-034).

References

- Chai, J., Chen, W., Zhu, Y., Yao, Z.-X., and Zhao, D. (2023). A hierarchical deep reinforcement learning framework for 6-dof ucav air-to-air combat. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 53(9):5417–5429.
- Chen, X., Wang, C., Zhou, Z., and Ross, K. (2021). Randomized ensemble double q-learning: Learning fast without a model. *arXiv preprint arXiv:2101.05982*.
- Cheng, C.-A., Kolobov, A., and Swaminathan, A. (2021). Heuristic-guided reinforcement learning. In Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W., editors, *Advances in Neural Information Processing Systems*, volume 34, pages 13550–13563. Curran Associates, Inc.

- De Marco, A., D’Onza, P. M., and Manfredi, S. (2023). A deep reinforcement learning control approach for high-performance aircraft. *Nonlinear Dynamics*, 111(18):17037–17077.
- Duan, J., Wang, W., Xiao, L., Gao, J., Li, S. E., Liu, C., Zhang, Y.-Q., Cheng, B., and Li, K. (2025). Distributional soft actor-critic with three refinements. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(5):3935–3946.
- Hu, D., Yang, R., Zuo, J., Zheng, W., Zhao, Y., and Zhang, Q. (2021). Intelligent maneuvering decision of unmanned combat aircraft based on lstm-dueling dqn. *Tactical Missile Technol*, 6:97–104.
- HU, Z., LIANG, X., ZHANG, J., YOU, X., and MA, C. (2025). Exploring crash induction strategies in within-visual-range air combat based on distributional reinforcement learning. *Chinese Journal of Aeronautics*, 38(9):103663.
- Kuznetsov, A., Shvechikov, P., Grishin, A., and Vetrov, D. (2020). Controlling overestimation bias with truncated mixture of continuous distributional quantile critics. In *International conference on machine learning*, pages 5556–5566. PMLR.
- Li, B., Huang, J., Bai, S., Gan, Z., Liang, S., Evgeny, N., and Yao, S. (2023). Autonomous air combat decision-making of uav based on parallel self-play reinforcement learning. *CAAI Transactions on Intelligence Technology*, 8(1):64–81.
- MC, Y. (2024). Intelligent air combat algorithm and simulation research based on reinforcement learning. In *International conference on machine learning*, pages 46–50. Xi’an: Northwestern Polytechnical University.
- Pope, A. P., Ide, J. S., Mićović, D., Diaz, H., Rosenbluth, D., Ritholtz, L., Twedt, J. C., Walker, T. T., Alcedo, K., and Javorsek, D. (2021). Hierarchical reinforcement learning for air-to-air combat. In *2021 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 275–284.
- Pope, A. P., Ide, J. S., Mićović, D., Diaz, H., Twedt, J. C., Alcedo, K., Walker, T. T., Rosenbluth, D., Ritholtz, L., and Javorsek, D. (2023). Hierarchical reinforcement learning for air combat at darpa’s alphadogfight trials. *IEEE Transactions on Artificial Intelligence*, 4(6):1371–1385.

- Qian, C., Zhang, X., Li, L., Zhao, M., and Fang, Y. (2024). H3e: Learning air combat with a three-level hierarchical framework embedding expert knowledge. *Expert Systems with Applications*, 245:123084.
- Qiu, X., Yao, Z., Tan, F., Zhu, Z., and Lu, J.-G. (2020). One-to-one air-combat maneuver strategy based on improved td3 algorithm. In *2020 Chinese Automation Congress (CAC)*, pages 5719–5725.
- Shan, S., Yang, M., Zhang, W., and Gao, C. (2022). Continuous decision-making method for autonomous air combat. *Adv. Aeronaut. Sci. Eng*, 13:47–58.
- Wang, X., Wang, Y., Su, X., Wang, L., Lu, C., Peng, H., and Liu, J. (2024). Deep reinforcement learning-based air combat maneuver decision-making: literature review, implementation tutorial and future direction. *Artificial Intelligence Review*, 57(1):1.
- Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., and Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pages 1995–2003. PMLR.
- Xu, J., Guo, Q., Xiao, L., Li, Z., and Zhang, G. (2019). Autonomous decision-making method for combat mission of uav based on deep reinforcement learning. In *2019 IEEE 4th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*, volume 1, pages 538–544.
- Xu, J., Zhang, J., Yang, L., and Liu, C. (2022). Autonomous decision-making for dogfights based on a tactical pursuit point approach. *Aerospace Science and Technology*, 129:107857.
- Xu, X., Wang, Y., Guo, X., Huang, K., and Zhang, X. (2025). Multi-uav air combat cooperative game based on virtual opponent and value attention decomposition policy gradient. *Expert Systems with Applications*, 267:126069.
- Yoo, J., Seong, H., Shim, D. H., Bae, J. H., and Kim, Y.-D. (2022). Deep reinforcement learning-based intelligent agent for autonomous air combat. In *2022 IEEE/AIAA 41st Digital Avionics Systems Conference (DASC)*, pages 1–9.
- ZHANG, J., WANG, D., YANG, Q., SHI, Z., JI, L., SHI, G., and WU, Y. (2024). Loyal wingman task execution for future aerial combat: A hierarchical prior-based reinforcement learning approach. *Chinese Journal of Aeronautics*, 37(5):462–481.

Zhou, K., Wei, R., Zhang, Q., and Xu, Z. (2020). Learning system for air combat decision inspired by cognitive mechanisms of the brain. *IEEE Access*, 8:8129–8144.

Zhou, P., Huang, J., Zhang, S., Liu, G., Shu, B., and Tang, J. (2023). Intelligent air combat decision making and simulation based on deep reinforcement learning. *Acta Aeronautica et Astronautica Sinica*, 44(4):126731.

Appendix A. State space symbol description

In the state space of the MDP model for higher-level air combat decision-making, as shown in Eq. (1), lon_t^k , lat_t^k , and h_t^k denote the longitude, latitude, and altitude, respectively; $v_t^{k,east}$, $v_t^{k,north}$, and $v_t^{k,up}$ represent the velocity vectors in the east, north, and up directions, respectively; M_t^k denotes the Mach number; $a_{x,t}^k$, $a_{y,t}^k$, and $a_{z,t}^k$ represent the acceleration along the three axes in the body coordinate system; θ_t^k , φ_t^k , and γ_t^k represent the pitch, yaw, and roll angles of the flight path, respectively; $\omega_t^{k,\psi}$, $\omega_t^{k,\theta}$, and $\omega_t^{k,\gamma}$ are the angular velocities; α_t^k and β_t^k represent the aerodynamic angle of attack and sideslip angle, respectively; θ_t^b and φ_t^b represent the pitch and yaw angles of the opposing aircraft's position in the body coordinate system, respectively; l_t^b denotes the distance between the opposing aircraft and the friendly aircraft; $\Delta v_{x,t}^b$, $\Delta v_{y,t}^b$, $\Delta v_{z,t}^b$, and ΔM_t represent the three-axis components of the opposing aircraft's velocity and Mach number relative to the friendly aircraft in the body coordinate system; $n_{missile}$ represents the remaining number of missiles.

In the state space of the MDP model for lower-level flight control, as shown in Eq. (3), lon_t , lat_t and h_t denote longitude, latitude and altitude, respectively; v_t^{east} , v_t^{north} and v_t^{up} are the velocity components in the east, north and up directions; v_t is the speed magnitude; $a_t^{x^b}$, $a_t^{y^b}$ and $a_t^{z^b}$ are the linear accelerations along the three axes of the body coordinate system; θ_t , φ_t and γ_t are the flight-path pitch, heading and roll angles; $\omega_t^{x^b}$, $\omega_t^{y^b}$ and $\omega_t^{z^b}$ are the angular velocity components about the three body axes; α_t and β_t are the aerodynamic angle of attack and sideslip angle; Δv_t , $\Delta \theta_t$, $\Delta \varphi_t$ represent the speed, flight-path pitch and heading angles that need to be adjusted.

Appendix B. Self-play training protocol for R2SP

The dynamic and adversarial nature of WVR air combat calls for a training scheme that can continuously adapt to evolving opponent strategies (Li et al.,

2023). R2SP adopts a self-play framework tailored to the proposed architecture, as illustrated conceptually in Fig. B.11. The self-play process is divided into two key phases: a *training phase* and an *evaluation phase*.

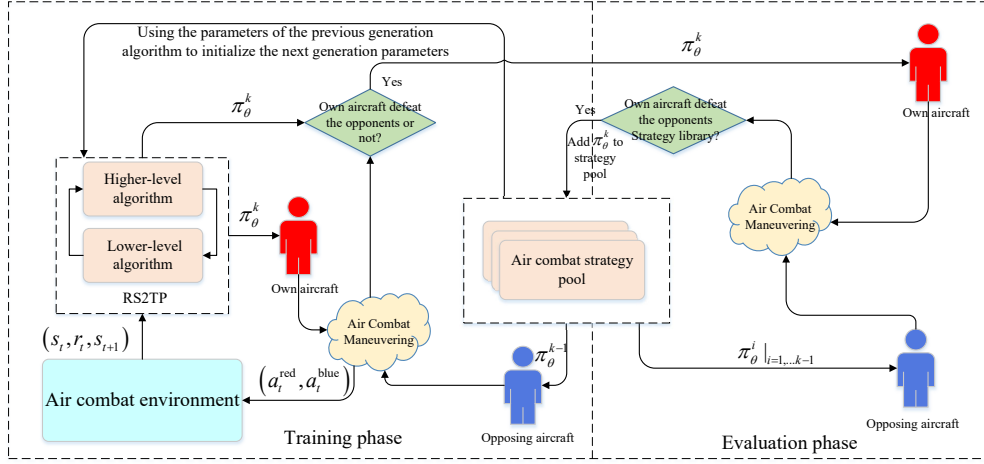


Figure B.11: Air combat self-play process setup.

Training phase. In the training phase, the R2SP agent interacts with the WVR air combat environment against a set of opponents sampled from a *strategy pool*. The strategy pool initially contains a small number of reference opponents (e.g., rule-based tactics or the first-generation learned policy). During this phase,

- The own aircraft (red side) is controlled by the current R2SP policy $\pi_{\theta}^{(k)}$ of generation k .
- The opponent (blue side) is sampled from the existing strategy pool $\mathcal{P}^{(k)} = \{\pi^{(0)}, \pi^{(1)}, \dots, \pi^{(k-1)}\}$.
- The environment returns trajectories τ , from which the high-level and low-level R2SP components are updated using the mechanisms in Sec.s 4.2 and 4.3, including trajectory-level reward relabelling and progressive sub-task guided control.

The training phase continues until pre-defined convergence or performance conditions are met. A typical stopping rule is that the estimated win rate of $\pi_{\theta}^{(k)}$ against the current opponent mixture exceeds a threshold (e.g., 30%) with sufficient confidence, at which point the system transitions to the evaluation phase.

Evaluation phase. In the evaluation phase, the opponent sequentially adopts all prior generations of R2SP strategies drawn from the strategy pool $\mathcal{P}^{(k)}$. The goal is to test whether the newly trained strategy $\pi_{\theta}^{(k)}$ remains robust against a diverse set of historical opponents. Concretely,

- For each $\pi^{(j)} \in \mathcal{P}^{(k)}$, a number of WVR engagements are conducted between $\pi_{\theta}^{(k)}$ (red side) and $\pi^{(j)}$ (blue side).
- The empirical win/tie/loss rates are recorded, and a generation-level performance metric is computed (e.g., average win rate over the pool or the worst-case win rate against any pool member).

If $\pi_{\theta}^{(k)}$ consistently outperforms the strategies in the pool according to a specified criterion (e.g., exceeding a target win rate against all historical generations), the new strategy is added to the pool,

$$\mathcal{P}^{(k+1)} \leftarrow \mathcal{P}^{(k)} \cup \{\pi_{\theta}^{(k)}\},$$

and the system enters the next training phase using this updated pool. Otherwise, additional training is carried out against the most challenging opponents (e.g., those for which the current generation exhibits the lowest win rate) until the evaluation criterion is satisfied.

Overall effect. By iterating between training and evaluation phases, the self-play framework encourages the R2SP policy to evolve continuously in response to increasingly strong adversaries. Coupled with trajectory-level reward relabelling at the tactical level and progressive sub-task guided control at the execution level, this leads to robust and tactically meaningful WVR air combat behaviours, without requiring handcrafted opponent models or manually designed tactics.

Appendix C. Missile modeling

The missile system is decomposed into three primary components: a missile motion model, an overload control (guidance) model, and a seeker detection model.

Appendix C.1. Conditions of missile launch

After the fire-control radar locks onto an enemy aircraft, missile launch is allowed only when several conditions are simultaneously satisfied:

1. the target lies within the maximum effective range of the missile;
2. the launching aircraft still has at least one missile remaining;
3. the radar has maintained a stable track on the hostile target for more than 10 consecutive time steps;
4. no other missile is currently targeting the same opposing aircraft in the present engagement.

This design encourages tactically sound missile employment and prevents wasteful or redundant launches.

Appendix C.2. Missile motion model

The missile is conceptualised as a rigid body with three translational degrees of freedom. In a local navigation frame, its kinematic equations of motion can be written as

$$\dot{x}_m = V_m \cos \gamma_m \cos \chi_m, \quad (\text{C.1})$$

$$\dot{y}_m = V_m \cos \gamma_m \sin \chi_m, \quad (\text{C.2})$$

$$\dot{z}_m = -V_m \sin \gamma_m, \quad (\text{C.3})$$

where (x_m, y_m, z_m) denote the missile position, V_m is the speed, γ_m is the flight-path elevation angle and χ_m is the heading angle.

The corresponding dynamic equations are

$$\dot{V}_m = \frac{1}{m_m} (P_x - X_D) - g \sin \gamma_m, \quad (\text{C.4})$$

$$\dot{\gamma}_m = \frac{n_y}{V_m} - \frac{g}{V_m} \cos \gamma_m, \quad (\text{C.5})$$

$$\dot{\chi}_m = \frac{n_z}{V_m \cos \gamma_m}, \quad (\text{C.6})$$

where m_m denotes the missile mass, P_x is the thrust component along the velocity direction, X_D denotes aerodynamic drag, g is gravitational acceleration, and n_y, n_z are the normal overload components in the pitch and yaw channels, respectively. The overloads (n_y, n_z) are controlled by the guidance law described below, and are subject to physical constraints imposed by the missile airframe and control surfaces.

Appendix C.3. Overload control (guidance) model

Let $\mathbf{R} = \mathbf{r}_{\text{tar}} - \mathbf{r}_{\text{m}}$ denote the relative position vector from the missile to the target, and let λ be the line-of-sight (LOS) angle between the missile velocity vector and $\mathbf{R}$. A proportional navigation (PN) guidance law is used in this study to generate the commanded normal overload,

$$\mathbf{n}_c = NV_c \dot{\lambda}, \quad (\text{C.7})$$

where N is the navigation ratio, V_c is the closing velocity magnitude and $\dot{\lambda}$ is the LOS rate vector. The commanded overload $\mathbf{n}_c$ is then mapped to the pitch and yaw channels,

$$n_{y,c} = \mathbf{n}_c \cdot \mathbf{e}_y, \quad n_{z,c} = \mathbf{n}_c \cdot \mathbf{e}_z, \quad (\text{C.8})$$

where $\mathbf{e}_y, \mathbf{e}_z$ are the unit vectors of the lateral axes in the body or velocity frame.

To account for actuator dynamics and structural limits, the actual overloads (n_y, n_z) follow first-order lag dynamics with saturation:

$$\dot{n}_y = \frac{1}{T_n} (n_{y,c} - n_y), \quad (\text{C.9})$$

$$\dot{n}_z = \frac{1}{T_n} (n_{z,c} - n_z), \quad (\text{C.10})$$

subject to

$$|n_y| \leq n_{y,\max}, \quad |n_z| \leq n_{z,\max}, \quad (\text{C.11})$$

where T_n is a time constant and $n_{y,\max}, n_{z,\max}$ are the maximum allowable overloads in the pitch and yaw channels. The corresponding maximum achievable angular rates in (C.5)–(C.6) follow directly from substituting $n_{y,\max}$ and $n_{z,\max}$ into the dynamic equations.

Appendix C.4. Seeker detection model

The missile seeker imposes a detection range constraint on the engagement. For an infrared seeker, the maximum detection distance $R_{\max}$ can be approximated by a radiative transfer relationship of the form

$$R_{\max} = \left(\frac{\varepsilon \sigma T_{\text{tar}}^4 A_{\text{tar}} A_{\text{sen}} \eta_{\text{air}} \eta_{\text{sen}}}{4\pi P_{\text{sen}}^{\max}} \right)^{1/2}, \quad (\text{C.12})$$

where

- ε denotes the emissivity of the target aircraft;

- $\sigma = 5.67 \times 10^{-8} \text{ W}/(\text{m}^2\text{K}^4)$ is the Stefan–Boltzmann constant;
- T_{tar} is the effective surface temperature of the target;
- A_{tar} is the effective radiating area of the target;
- A_{sen} is the effective aperture area of the seeker;
- η_{air} is the atmospheric attenuation coefficient;
- η_{sen} is the detection efficiency of the seeker;
- $P_{\text{sen}}^{\text{max}}$ is the minimum detectable signal power at the seeker receiver.

In addition, modern infrared missiles with off-boresight launch capability require stable target acquisition prior to launch. In this work, stable target acquisition is defined as the fire-control radar (FCR) maintaining lock-on for at least 0.5 s. If, under this condition, the interceptor–target distance satisfies (C.12), the missile launch criterion is deemed to be met and weapon release is authorised.

Appendix D. Radar modeling

The aircraft’s fire-control radar is a key component in aerial combat, enabling precise target lock-on and continuously transmitting target data to the missile. Consequently, the fidelity of its model significantly influences the realism of operational simulations. This appendix outlines the detection equation, search modes and target acquisition logic adopted in this study.

Appendix D.1. Radar detection equation

A simplified monostatic pulse-radar range equation is used to model the maximum detection range:

$$R_{\text{max}} = \left(\frac{P_{\text{t}} G_{\text{t}}^2 \lambda^2 \sigma}{(4\pi)^3 P_{\text{min}}} \right)^{1/4}, \quad (\text{D.1})$$

where

- R_{max} is the maximum detection range;
- P_{t} is the transmitter power;
- G_{t} is the antenna transmit gain;

- λ is the radar wavelength;
- σ is the target radar cross-section (RCS);
- $P_{\min}$ is the minimum detectable signal power at the receiver.

In the context of aerial combat, let R denote the distance between the centroids of the two engaged aircraft. An opposing aircraft is considered to be within the radar detection envelope if and only if $R \leq R_{\max}$.

Appendix D.2. Search modes and scanning patterns

The radar system can operate in several modes (air-to-air, maritime, ground attack). Since this study focuses on one-versus-one WVR air combat, we confine attention to the air-to-air engagement mode, which includes three search sub-modes: horizontal search, vertical search and fixed-axis search (MC, 2024). The corresponding search volumes are given in Table D.3.

Table D.3: Airborne radar search modes and scanning patterns.

Sub-mode	Horizontal azimuth	Vertical azimuth	Boresight offset	Search time
Horizontal search	$-10^\circ \sim 10^\circ$	$-15^\circ \sim 5^\circ$	$\pm 10^\circ$	3 s
Vertical search	$-5^\circ \sim 5^\circ$	$-10^\circ \sim 30^\circ$	$\pm 5^\circ$	3 s
Fixed-axis search	$-1^\circ \sim 1^\circ$	$-1.5^\circ \sim 0.5^\circ$	0°	0.01 s

Appendix D.3. Target acquisition logic

Once the own aircraft has selected the radar operational mode and scanning pattern, a target aircraft must simultaneously satisfy the following conditions to be acquired by the radar:

1. **Range constraint:** The slant range between the own aircraft and the target must be less than the radar acquisition range $R_{\text{acq}} \leq R_{\max}$.
2. **Angular constraint:** The target must lie within the current radar scanning volume (horizontal and vertical azimuth limits) associated with the selected search mode.
3. **LOS-rate constraint:** The line-of-sight (LOS) rate between the own aircraft and the target must remain within the radar's tracking limits, i.e.,

$$|\dot{\lambda}_{\text{az}}| \leq \dot{\lambda}_{\text{max}}^{\text{az}}, \quad |\dot{\lambda}_{\text{el}}| \leq \dot{\lambda}_{\text{max}}^{\text{el}}.$$

4. **Radial-velocity constraint:** The target's radial velocity must fall within the radar's detectable speed gate.

Figure D.12 illustrates the target acquisition process.

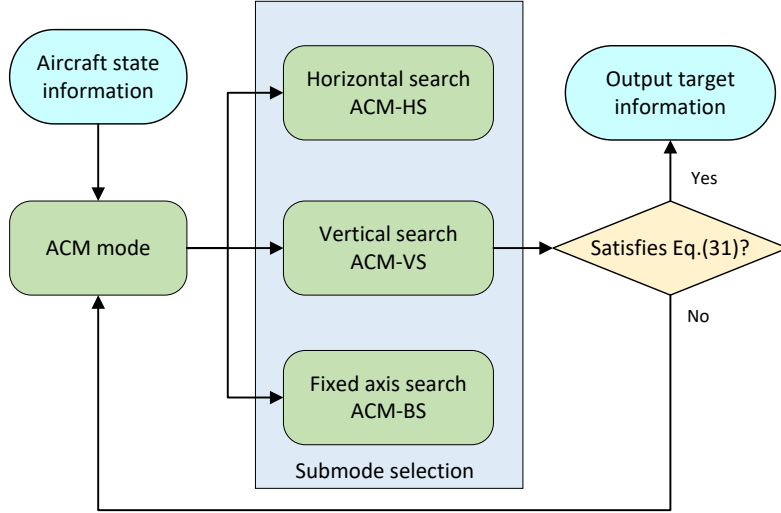


Figure D.12: Airborne radar target acquisition process.

Let $\mathbf{V}_{\text{rel}}$ denote the relative velocity between the target and the own aircraft, and $\mathbf{R}$ the relative position vector. Following the Doppler principle, the radial (LOS) component of the relative velocity and the absolute target speed are given by

$$V_{\Delta}^b = \frac{\mathbf{V}_{\text{rel}} \cdot \mathbf{R}}{\|\mathbf{R}\|}, \quad (\text{D.2})$$

$$V_{\text{abs}}^b = \|\mathbf{V}_{\text{tar}}\|, \quad (\text{D.3})$$

where $\mathbf{V}_{\text{tar}}$ is the target velocity vector. Due to the echo-processing frequency band limits, the radial velocity must satisfy

$$V_{\min}^b \leq V_{\Delta}^b \leq V_{\max}^b, \quad (\text{D.4})$$

where $V_{\max}^b$ and $V_{\min}^b$ are the maximum and minimum permissible radial velocities. To mitigate interference from low-speed environmental clutter, the target's absolute speed must also exceed a clutter-rejection threshold,

$$V_{\text{abs}}^b \geq V_{\text{clutter}}. \quad (\text{D.5})$$

Only when all of the above range, angular, LOS-rate and velocity-gate conditions are jointly satisfied is the target declared acquired by the radar. Subsequent track maintenance and lock-on logic then follow the criteria given in Sec. 3.2.3.

Appendix E. R2SP algorithm performance parameters

This appendix summarises the key hyperparameters used by the R2SP framework for WVR air combat. For clarity, we separate them into three groups: common DRL parameters, lower-level flight-control parameters and higher-level engagement parameters.

Appendix E.1. Common DRL parameters for aerial combat

Table E.4: Common DRL parameters for aerial combat (shared by R2SP and baselines).
Note: please check numeric values against your original code or notes and adjust if necessary.

Parameter	Value
Actor network layers	3
Actor hidden layer sizes	256, 256, 128, 128
Actor learning rate	3×10^{-4}
Critic network layers	3
Critic hidden layer sizes	256, 256, 128, 128
Critic learning rate	3×10^{-4}
Discount factor γ	0.99
Batch size b_{sc}	256
Target-network update rate τ	5×10^{-3}
Target entropy (SAC)	auto-tuned
Simulation step size	0.01 s
Testing decision interval	0.4 s

Appendix E.2. Lower-level R2SP parameters for aerial combat

Appendix E.3. Higher-level R2SP parameters for aerial combat

Appendix F. Pseudocode design

Appendix F.1. Trajectory-level reward relabeling

The overall higher-level R2SP algorithm, including reward relabelling, buffer updates and parameter learning, is summarised in Algorithm 1.

Table E.5: R2SP lower-level (flight control) parameters for aerial combat.
Note: values reconstructed from the PDF; please confirm from original.

Parameter	Value
Number of sub-tasks N_{count}	3 (pitch / heading / speed)
Stage length N_{min} (steps per sub-task stage)	600
Minimum sub-task weight $w_{k,\text{min}}$	0.1
Maximum sub-task weight $w_{k,\text{max}}$	1.0
Achievement threshold N_{thr}	20
Backtracking threshold count N'_{thr}	5
Performance threshold α_{low}	0.1
Max episode length (low-level steps)	1,200
Replay buffer size (low-level)	1×10^6

Table E.6: R2SP higher-level (engagement decision) parameters for aerial combat.
Note: mapping of numeric values to symbols follows Sec. 4.2; verify against your implementation.

Parameter	Value
Initial sampling ratio λ_{beg}	0.3
Final sampling ratio λ_{end}	0.7
High-quality buffer size $\mathcal{B}_{\text{exp}}$	5×10^5
Online buffer size $\mathcal{B}_{\text{online}}$	5×10^5
Episode buffer capacity $\mathcal{B}$	3,000
Pre-training steps using heuristic policy T_{exp}	1.2×10^5
Start of sampling-ratio schedule T_{beg}	1.2×10^5
End of sampling-ratio schedule T_{end}	1.2×10^6
Threshold factor α (Eq. (19))	0.5
Evaluation interval (episodes per evaluation)	2,000

Appendix F.2. Progressive sub-task guided

The complete lower-level flight control method of R2SP, including sub-task scheduling, weight updates and network training, is summarised in Algorithm 2.

Appendix G. Rule-based policy

In WVR air combat, a basic tactical objective is to align one's own aircraft towards the opponent's rear hemisphere while avoiding being placed in the opponent's firing envelope. First, we briefly recall the relevant definitions. The angle of attack (AoA) is the angle between the aircraft's velocity vector and its longitudinal axis. The aspect angle (AA) is the angle between the line of sight to the target and the target's velocity vector. The heading crossing angle (HCA) is the difference between the headings of the two aircraft projected onto the horizontal plane. These angles are illustrated in Fig. G.13. Based on AoA, AA and HCA, we derive a set of rule-based heading adjustment commands that tend to bring the fighter into a favourable firing position. The corresponding heading adjustment can be expressed in Eq. G.1

$$\begin{cases} \vec{R}^b = L_g^b \vec{R} \\ \Delta\theta_t^b = \arcsin \left(z_{\vec{R}_b} / \left| \vec{R}^b \right| \right) \\ \Delta\varphi_t^b = \arcsin \left(y_{\vec{R}_b} / \left| \vec{R}^b \right| \cos(\theta) \right) \end{cases} \quad (\text{G.1})$$

where L_g^b is the transformation matrix from the ground coordinate system to the body coordinate system; $\vec{R}$ and $\vec{R}_b$ denote the distance vectors from own aircraft to the opposing aircraft in the ground and body coordinate systems, respectively; $\Delta\theta_t^b$ and $\Delta\varphi_t^b$ represent the required pitch and heading angle adjustments.

Algorithm 1 R2SP Higher-Level Algorithm for Air Combat Decision-Making

```
1: Initialize the actor network  $\pi_\theta$  and target network  $\pi'_\theta$ 
2: Initialize critic networks  $Q_{\omega_1}, Q_{\omega_2}$  and target networks  $Q'_{\omega_1}, Q'_{\omega_2}$ 
3: Initialize replay buffers  $\mathcal{B}, \mathcal{B}_{\text{online}}, \mathcal{B}_{\text{exp}}$ 
4: Set sampling proportion  $\lambda_t$ , batch size  $c_{\text{bs}}, t_{\text{beg}}, T_{\text{max}}^{\text{ep}}$ 
5: for each episode do
6:   for  $t = 0$  to  $T_{\text{max}}^{\text{ep}}$  do
7:     while not done do
8:       if  $t < t_{\text{beg}}$  then
9:         Calculate  $a_t$  by heuristic strategy
10:      else
11:         $a_t = \pi_\theta$ 
12:      end if
13:       $r_t, s_{t+1}, \text{done} = env.\text{step}(a_t)$ 
14:      Add  $(s_t, a_t, r_t, s_{t+1})$  to  $\mathcal{B}$ 
15:      if  $\text{len}(\mathcal{B}_{\text{exp}}) > \lambda_t c_{\text{bs}}$  and  $\text{len}(\mathcal{B}_{\text{online}}) > (1 - \lambda_t) c_{\text{bs}}$  then
16:        Sample  $\lambda_t c_{\text{bs}}$  data  $A$  from  $\mathcal{B}_{\text{exp}}$ 
17:        Sample  $(1 - \lambda_t) c_{\text{bs}}$  data  $A'$  from  $\mathcal{B}_{\text{online}}$ 
18:        Train  $Q_{\omega_1}, Q_{\omega_2}, \pi_\theta$  using  $A, A'$ 
19:        Update target networks  $Q'_{\omega_1}, Q'_{\omega_2}, \pi'_\theta$ 
20:      end if
21:    end while
22:    Relabel rewards in  $\mathcal{B}$  by Eq. (10)
23:    if  $t < t_{\text{beg}}$  then
24:      Add relabeled  $\mathcal{B}$  to  $\mathcal{B}_{\text{exp}}$ 
25:    else
26:      Compute trajectory reward  $R(\tau)$  and add to  $\mathcal{B}_{\text{ep}}$ 
27:      if Eq. (19) is workable then
28:        Add relabeled  $\mathcal{B}$  to  $\mathcal{B}_{\text{exp}}$ 
29:      else
30:        Add relabeled  $\mathcal{B}$  to  $\mathcal{B}_{\text{online}}$ 
31:      end if
32:    end if
33:    Update  $\lambda_t$  by Eq. (20)
34:  end for
35: end for
```

Algorithm 2 R2SP Lower-Level Algorithm for Flight Control

```
1: Initialize the actor network  $\pi_\theta$  and target network  $\pi'_\theta$ 
2: Initialize critic networks  $Q_{\omega_1}, Q_{\omega_2}$  and target networks  $Q'_{\omega_1}, Q'_{\omega_2}$ 
3: Initialize replay buffer  $\mathcal{B}$ 
4: Initialize weight matrix  $\mathbf{W}_g^j$  and reward function  $R(t)$ 
5: Set  $N_{\text{thr}}, \tau_{\text{thr}}^j, N_{\text{count}}$ , and initialize  $n_{\text{cur}} = 0$ , sub-task  $j = 0$ 
6: for each episode do
7:   while not done do
8:      $a_t = \pi_\theta(s_t)$ 
9:     Get  $(s_{t+1}, \text{done})$  from environment using  $(s_t, a_t)$ 
10:    Calculate  $r_t(s_t, a_t, s_g^j)$  by Eq. (22)
11:    Add  $(s_t, a_t, r_t, s_{t+1})$  to  $\mathcal{B}$ 
12:    Train  $Q_{\omega_1}, Q_{\omega_2}, \pi_\theta$  using  $\mathcal{B}$  with losses  $L_Q(\omega_1), L_Q(\omega_2), L_x(\theta)$ 
13:    Update target networks  $Q'_{\omega_1}, Q'_{\omega_2}, \pi'_\theta$ 
14:  end while
15:  if  $R_n^j \geq \tau_{\text{thr}}^j$  then
16:    Calculate  $\mathbf{W}_g^j$  by Eq. (24) and Eq. (25)
17:     $\delta = 0$ 
18:     $n_{\text{cur}} = n_{\text{cur}} + 1$ 
19:    if  $n_{\text{cur}} > N_{\text{count}}$  then
20:      sub-task  $j = j + 1$ 
21:      Update weight matrix  $\mathbf{W}_g^j$  by Eq. (23)
22:    else
23:      if  $\delta > N_{\text{thr}}$  then
24:         $n_{\text{cur}} = n_{\text{cur}} - 1$ 
25:        Update weight matrix  $\mathbf{W}_g^j$  by Eq. (23)
26:         $\delta = 0$ 
27:      else
28:         $\delta = \delta + 1$ 
29:      end if
30:    end if
31:  end if
32: end for
```

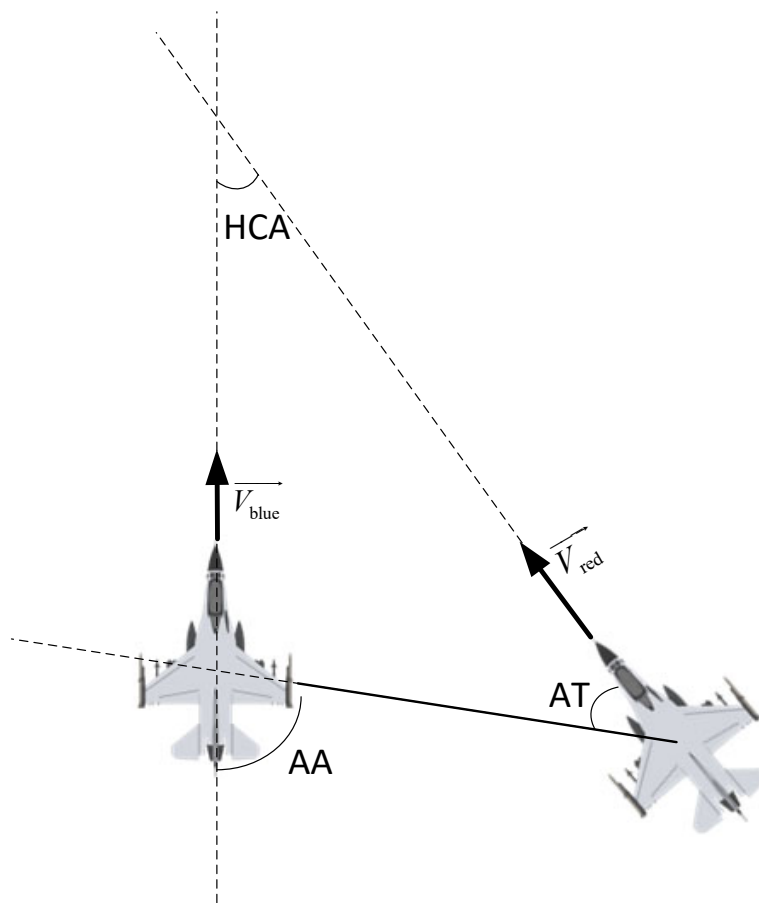


Figure G.13: R2SP algorithm architecture.